Facetwork

A Language-Directed, Lock-Free Model for Live-Updatable Distributed Workflow Execution

A dissertation submitted in partial fulfilment of the requirements for the degree of Doctor of Philosophy.

Candidate: Claude (Opus 4.6, 1M context) — Department of Distributed Systems and Programming Languages.

Supervisor: Ralph Lemke.

Abstract

Distributed workflow systems sit at the uncomfortable intersection of three ongoing arguments in systems research: how work should be *described* (declaratively, imperatively, or somewhere in between), how work should be *coordinated* (through a central scheduler, a peer-to-peer claim protocol, or a broker), and how work should be *recovered* after faults (through replay of an event log, through persisted state, or through manual intervention). Each of the dominant industrial systems resolves these tensions differently. Temporal and Cadence treat workflows as deterministic programs whose history is the source of truth; Camunda models them as BPMN graphs whose state lives in a relational database; Jenkins treats them as imperative pipelines anchored to a single controller; Argo treats them as Kubernetes-native YAML. Each resolution has paid for its strengths with specific, structural weaknesses.

This dissertation describes **Facetwork**, a distributed workflow runtime that takes a different, and I will argue better, position on all three axes. Facetwork introduces a declarative domain-specific language, the **Facetwork Flow Language** (FFL), in which domain experts describe workflows as algebras of *facets* — typed units of computation composable via `andThen`, mixins, implicit parameters, and conditional branching — without ever touching the runtime. Facetwork coordinates work through a **lock-free claim protocol** over an atomic document store, leveraging partial unique indices and document-level compare-and-swap semantics to achieve exactly-once execution without a centralised lock manager or coordinator. Facetwork recovers from faults through a **persistent execution graph**, not a replayed event log, allowing individual steps to be re-run, reset, or repaired in place; this drops the determinism constraint that handicaps replay-based systems and admits long-lived, non-idempotent, externally-observable side effects as first-class citizens. Finally, Facetwork is **live-updatable**: new handlers can be registered at runtime, stages within a single long-running task can carry their own dynamic timeout budgets, and individual runners can be drained or quarantined without interrupting the fleet.

The contributions of this thesis are:

1. A type-annotated, LALR-parsable DSL (FFL) in which workflow topology, typed data contracts, mixin composition, and event facets are all first-class citizens, with a clean syntactic and semantic separation between *what* a workflow does (authored by domain programmers) and *how* each facet is implemented (authored by service-provider programmers).
2. A formal account of the lock-free claim protocol used by Facetwork, including its correctness argument against the Fischer-Lynch-Paterson impossibility, a derivation of its liveness properties under lease-based leadership, and a reclaim rule that guarantees progress despite arbitrary handler crashes without compromising safety.
3. A **staged timeout model** for long-running handlers that generalises the flat execution-timeout used by most workflow systems into a nested, dynamically extensible budget. The stage model composes cleanly with the runner's global watchdog, retains a hard safety ceiling, and supports input-size-aware budgets for realistic workloads such as bulk geospatial imports.
4. A comparative analysis of Facetwork against four representative systems — Temporal (event-sourced workflow-as-code), Camunda (BPMN process engine), Apache Airflow (Python DAG scheduler), and Jenkins (centralised CI/CD controller) — on seven dimensions: topology description, coordination, recovery, updatability, observability, operational cost, and domain accessibility.
5. Evidence from a production OSM geocoder workload that the combined design yields multi-hour distributed imports whose recovery is incremental, whose handlers can be updated during execution, and whose failures are tractable at step granularity rather than whole-workflow granularity.

Facetwork is not the first distributed workflow system. It is, however, the first to simultaneously take a declarative-DSL stance on *description*, a lock-free document-atomic stance on *coordination*, and a live-state stance on *recovery and updatability*, and to show that the three choices reinforce rather than undercut each other.

Preface

I was asked to write a thesis on Facetwork by my supervisor. What follows is, I hope, not simply a description of the system but an argument — occasionally a polemic — for the design choices behind it. I have tried to be fair to the alternative systems I discuss. I have almost certainly failed in places, and I apologise in advance to any of their authors who read this and feel misrepresented. Where I have drawn contrasts sharply, it is because I believe the contrasts matter and that soft-pedalling them would make the thesis less, not more, useful.

I have written this thesis from inside the project. That is a methodological hazard I cannot fully mitigate. The reader should treat the design arguments as advocacy, the correctness arguments as claims open to formalisation, and the empirical claims as drawn from one specific, though non-trivial, workload. With those caveats, I begin.

Contents

Part I — Foundations

Chapter 1. Introduction

1.1 The three questions

A distributed workflow system must answer three questions, and almost every disagreement between systems can be traced back to a different combination of answers.

Q1. How is work described? Workflows must be written down somewhere. The notation chosen determines who can author them, what can be checked at compile time, what evolves easily under change, and what remains intelligible under incident response at three in the morning. The options span a spectrum: YAML DAGs (Airflow before its TaskFlow API, Argo, GitHub Actions), general-purpose code annotated with decorators (Prefect, Dagster, Temporal, Airflow TaskFlow), graphical standards (Camunda BPMN), and declarative DSLs (Facetwork's FFL, CWL, Nextflow).

Q2. How is work coordinated? Somewhere, some component must decide which worker executes which task. The options range from a single master scheduler (Jenkins, classical Airflow), through a brokered message queue (many Celery deployments), through event-sourced replay against a history service (Temporal), through Kubernetes itself (Argo), to peer-to-peer claim over a shared data store (Facetwork).

Q3. How is work recovered? When a worker dies halfway through a task, when the database flickers, when an external service rejects a request, what does the system do? The options are, broadly: (a) replay the event log deterministically (Temporal, Cadence), (b) persist state transitions in a database and retry from the last saved state (Camunda, Airflow, Facetwork), (c) require the pipeline author to encode idempotency manually (Jenkins, most CI/CD), or (d) abandon the run and restart from the top (the unstated default of too many systems).

No system gets all three questions right because the right answer depends on the workload. A payment flow with synchronous user-visible effects wants deterministic replay so that nothing is executed twice. A multi-hour geospatial import wants in-place retry of a single stage so that thirty-eight hours of prior work are not thrown away. A deployment pipeline wants a human to push a button and be told in plain language whether it succeeded.

Facetwork is not an attempt to be uniformly better than every other system at every workload. It is an attempt to be specifically better at a class of workloads poorly served by the dominant alternatives: **long-running, heterogeneous, evolving workflows authored by domain experts rather than platform engineers, coordinated across a fleet whose composition changes during execution, recovered at step granularity, and updated without stopping the fleet**. I will call this class **live long-running domain workflows** (LLDWs) and argue that it is both common and commercially underserved.

1.2 The thesis statement

I will defend the following claim:

> A distributed workflow system serving live long-running domain workflows should describe work declaratively in a typed DSL, coordinate work through lock-free atomic operations against a shared data store, persist execution state as a mutable graph rather than an event log, and expose fine-grained lifecycle and timeout controls to handlers and operators. Facetwork is such a system, and its design decisions reinforce each other: the declarative DSL enables compile-time topology checks that would be impossible in general-purpose code; the persisted graph makes step-level recovery tractable; the lock-free coordination admits runtime updates without fleet-wide synchronisation; and the live handler registration, in turn, makes the DSL's decoupling of topology and implementation operationally meaningful.

The remainder of this thesis substantiates this claim. Part II describes the Facetwork design; Part III compares it to representative alternatives; Part IV discusses limitations.

1.3 A motivating workload

Before the theory, an example. Consider importing OpenStreetMap data for a country into a PostGIS database as the first step in a geocoding pipeline. A typical workflow looks like this:

1. Resolve the download URL for the region.
2. Stream the PBF file (hundreds of megabytes to tens of gigabytes) to local storage.
3. Parse the PBF into staging tables on a disposable local PostgreSQL instance, tuned for bulk ingestion (`fsync=off`, `synchronous_commit=off`, `autovacuum=off`).
4. Merge the staging tables into the local main tables.
5. Transfer the result via `COPY` binary stream to the production PostgreSQL server.
6. Merge in batches into production tables, in a style that respects concurrent reads.
7. Write an audit log entry on the production server.
8. (Downstream) Index the data, render map tiles, extract amenities, compute statistics.

Stages (3) through (6) can run for **hours**. A naive workflow runtime configured with a single 15-minute task timeout kills the work long before it finishes. A runtime configured with a 48-hour timeout kills nothing at all and hides genuine stalls for up to two days. A runtime that requires the workflow author to predict wall-clock time up front is brittle: the same region loaded on a workstation and on a fleet node takes different times, and the same code on a larger region takes much longer.

What the workload actually wants is:

- A **per-stage** timeout budget, set by the handler at stage entry based on measured input size (PBF file bytes, row counts).
- A **hard global ceiling** that still catches runaway processes.
- **Heartbeats** from inside the stage so that genuine progress resets the watchdog, even when the overall stage budget is long.
- **Step-level recovery** so that a failure in stage (6) does not force the system to re-download the PBF.
- **Live updatability** of handlers so that a fix to the merge logic can be deployed to the fleet without cancelling the other twelve countries currently being imported.

The OSM workload will recur throughout the thesis as a concrete touchstone. The staged-timeout mechanism added to Facetwork in the course of this research (Chapter 8) was motivated directly by watching imports die at the global timeout just as staging merges were about to finish.

1.4 Contributions, restated

In brief:

1. **FFL as a flow algebra** (Chapter 4). An LALR-parsable, statically-typed DSL combining facet declarations, schema declarations, typed parameter lists, mixin composition, `andThen` sequencing (with parallel branches), conditional dispatch, `catch` blocks, `prompt` blocks, and in-situ `script` blocks. The grammar and AST are constrained so that workflow topology is statically checkable and handler identity is factored out of the description.
2. **A lock-free claim protocol** (Chapter 5). Built on atomic `find_one_and_update` over a MongoDB task collection, with a partial unique index on `(step_id, state=running)`, lease-based reclamation of dead tasks, and server orphan detection. I give a correctness argument for safety and a liveness argument under partial synchrony.
3. **Staged timeouts** (Chapter 8). A novel composable nesting of per-stage budgets inside global watchdogs, implemented as a `ctx.stage(name, timeout_ms=…)` context manager that extends the task's `stage_budget_expires` and lease. Budgets can be dynamically extended mid-stage when the handler discovers the input is larger than estimated.
4. **Live handler registration and fleet operations** (Chapter 6). Handlers are registered in the database rather than baked into the runner binary; runners are drained gracefully or quarantined at runtime without restart; rolling deploys are first-class operational procedures rather than implicit orchestration.
5. **A comparative framework** (Part III). I compare Facetwork against Temporal, Camunda, Airflow, and Jenkins on seven dimensions. The comparison is not neutral — I am arguing for Facetwork — but I have tried to present each alternative on its strongest ground.

Chapter 2. The Workflow Landscape

2.1 What is a workflow, exactly?

The word *workflow* has been stretched to cover almost any multi-step computation with persistence, which makes the landscape difficult to survey. For the purposes of this thesis I define a workflow system as software that provides all of the following:

1. A **description layer**: a notation in which multi-step computations (the *workflows*) are written down.
2. A **persistence layer**: storage of the current state of each in-flight workflow instance, durable across component restarts.
3. A **coordination layer**: a mechanism for choosing which worker executes which step at which time.
4. A **recovery layer**: a mechanism that deals with failures, whether of the worker, the coordinator, or the dependencies.
5. An **observability layer**: a way to inspect the state of each workflow instance, usually visual.

It is the interaction of these five layers that determines whether a system is pleasant or miserable to use on a given workload.

2.2 A taxonomy by description

Systems differ first in how they describe work. I distinguish four traditions:

The graphical standard tradition (BPMN-descended). Camunda, Flowable, Activiti. Workflows are drawn as diagrams that conform to the BPMN 2.0 standard; diagrams are stored as XML and executed by a process engine. The appeal is that business analysts can (in theory) draw flows that executives can read. The cost is that BPMN is a large, baroque standard with a long tail of edge-case elements (signal events, conditional intermediate throw events, non-interrupting boundary events), and that non-trivial BPMN diagrams are extremely hard to read in XML form, yet the XML form is the canonical one for version control.

The YAML-DAG tradition. Argo, GitHub Actions, GitLab CI. Workflows are static DAGs described in YAML, typically with templating. The appeal is that tooling — linters, policy scanners — operates on the YAML directly, and the YAML is canonical. The cost is that control flow more expressive than a DAG — loops bounded by runtime data, conditional fan-out, dynamic parallelism — falls out of the model, forcing either dynamic workflow generation at runtime (which loses the static-checking advantage) or grotesque YAML contortions (lookups that walk arrays by index in a templating language not designed for arrays).

The workflow-as-code tradition. Temporal, Prefect, Dagster, Airflow (post-TaskFlow), Cadence, Flyte. Workflows are written in a general-purpose programming language with framework-specific decorators and libraries. The appeal is access to the host language's ecosystem; control flow is whatever the host language supports. The cost is that the workflow definition is indistinguishable from its implementation: there is no separation between *what* the workflow is and *how* its steps are written, and workflows inherit all the non-determinism of the host language, which in turn forces determinism constraints if event-sourced replay is used for recovery.

The DSL tradition. CWL, Nextflow, Snakemake, Facetwork's FFL. Workflows are written in a purpose-built language whose grammar and types reflect workflow concepts. The appeal is that the language can enforce topology checks impossible in YAML (static type checking of parameters) or in general-purpose code (compile-time graph construction). The cost is that the DSL must be implemented, maintained, and documented, and its users must learn a new language whose community is by definition smaller than Python's or Java's.

2.3 A taxonomy by coordination

Systems differ second in how they decide which worker runs which task.

Centralised scheduler. Jenkins, classical Airflow with `SequentialExecutor` or `LocalExecutor`. A single controller process holds the queue, decides assignments, and dispatches work. The appeal is simplicity: one place to read, one place to debug. The cost is that the controller is a single point of failure and a scaling bottleneck. Systems in this tradition typically evolve towards executors (Airflow's `CeleryExecutor`, `KubernetesExecutor`) that push the actual dispatch to another mechanism while retaining the central scheduler.

Message broker. Celery + RabbitMQ, Sidekiq, many bespoke systems. Tasks are placed on a queue; workers pop them. The appeal is that the broker handles fanout and retry at the infrastructure level. The cost is that the broker becomes a new operational concern with its own failure modes, and that end-to-end task state is fragmented across the broker and the application database.

Event-sourced replay. Temporal, Cadence. A history service holds the ordered list of events that have happened to each workflow instance; workers replay that history to rebuild state, then execute the next decision. The appeal is that recovery is automatic and mathematically clean. The cost is that workflow code must be deterministic under replay, which excludes a wide class of practical operations unless laundered through special "activity" calls.

Kubernetes-native. Argo. Workflow steps are Kubernetes pods; coordination is done by a controller watching custom resources. The appeal is that Kubernetes provides scheduling, resource management, and isolation for free. The cost is that every step pays pod-startup latency (seconds at best) and every workflow depends on Kubernetes semantics.

Document-atomic claim. Facetwork, some bespoke Mongo- or Postgres-backed systems. Tasks are rows in a shared table; workers atomically claim them via compare-and-swap operations on the database. The appeal is that the database is the only stateful component; there is no coordinator, no broker, no lock service. The cost is that the database must support the required atomic operations efficiently, and that the claim protocol must be carefully designed to avoid both double-execution and starvation.

2.4 A taxonomy by recovery

Systems differ third in what they do when something breaks.

Event replay. Temporal, Cadence. Recovery is transparent from the workflow author's perspective — the system simply reconstructs state from history and continues — but the cost is determinism.

Persisted state with retries. Camunda, Airflow, Facetwork. The current state of each workflow instance is written to a database at every transition. After a failure, the system reads the state and picks up where it left off. Side effects are, by default, only at-least-once. Authors encode idempotency where needed.

Idempotency-by-design. Jenkins and much CI/CD. The author is expected to write pipelines that are safe to re-run from the top. Systems of this kind rely on the author's discipline, which is fine when the author is one platform engineer and awful when the author is a dozen teams with different assumptions.

Resume-from-checkpoint. Some data pipelines (Apache Flink, Apache Spark Structured Streaming). State is checkpointed periodically; on failure, the system rewinds to the last checkpoint and replays from there. This works well for streaming but awkwardly for workflows with external side effects.

2.5 Where Facetwork fits

Facetwork is a **DSL-described, document-atomic, state-persisted** workflow system with a distinctive emphasis on **live operational control** (drains, quarantines, rolling deploys, stage budgets). Its immediate competitors, one in each description tradition, are:

- Temporal (workflow-as-code, event-replay);
- Camunda (graphical, state-persisted);
- Airflow (code DAG, centralised scheduler + executors);
- Argo (YAML DAG, Kubernetes-native).

Jenkins is not a direct competitor — it is a CI/CD controller, not a workflow runtime — but its shape represents a pattern worth contrasting against because it is so widely familiar.

Chapter 3. Related Work

3.1 The theory of distributed coordination

The theoretical basis of Facetwork's coordination protocol stands on well-trodden ground. The foundational impossibility result is Fischer, Lynch, and Paterson's 1985 proof that no deterministic consensus protocol can guarantee termination in an asynchronous system with even one faulty process [FLP85]. This forces every practical distributed coordinator either to (a) accept non-termination in the worst case, (b) assume partial synchrony (as in Dwork, Lynch, and Stockmeyer's 1988 generalisation [DLS88]), or (c) reduce the problem to leader election and fall back to timeouts.

Lamport's Paxos [Lam98] and its modern descendant Raft [OO14] are the canonical partially-synchronous consensus protocols. Facetwork does not run consensus in the strict sense; it exploits the fact that the database it targets (MongoDB) already runs a replicated-state-machine protocol internally, offering linearisable single-document updates. Facetwork's contribution is to *project* the workflow coordination problem onto the database's atomic primitives so that no additional coordinator is needed.

Chubby [Bur06] and ZooKeeper [HKJR10] popularised the idea of using a separately-deployed coordination service for leases, leader election, and configuration. Both have been enormously influential, and both contribute to the operational complexity of the systems that use them. A theme of this thesis is that a system which can avoid a ZooKeeper dependency, while keeping the guarantees ZooKeeper provides, should avoid it.

Lease-based failure detection dates to Gray and Cheriton [GC89] in the context of distributed file caches. The idea — hold a lease with a timeout, renew it by heartbeat, and allow anyone to reclaim an expired lease — is ubiquitous now. Facetwork's lease scheme is a straightforward application of the classical model with two small refinements: leases are per-task rather than per-worker, and heartbeats carry progress data so that the stuck-task watchdog can distinguish slow-but-alive from genuinely stuck.

3.2 Workflow languages and DSLs

The BPEL/BPMN line of business-process languages [OMG11] is the most widely deployed workflow DSL family in industrial use. BPMN's graphical notation has made it an accepted interchange format for enterprise process modelling; its XML serialisation has made it universally verbose. BPEL, the executable form that preceded the BPMN 2.0 execution semantics, was explicitly designed for SOAP-era web services and inherits their ceremonious style.

Scientific workflow DSLs — CWL [CWL20], Nextflow [DTP+17], Snakemake [KR12] — take a narrower focus, emphasising data-dependency inference from input-output declarations and execution across heterogeneous compute backends. They have pushed the idea that a DSL for workflows should expose computation as functions with typed inputs and outputs, a design Facetwork adopts and generalises.

Musketeer [GSR+15], Dryad [IBY+07], Pig Latin [ORS+08], and Flume [CCH+10] lie on the borderline between workflow systems and data-processing frameworks. Each contributed to the idea that the user describes intent (a dataflow graph or relational algebra expression) while the system decides placement and scheduling — the same decoupling Facetwork enforces between domain-programmer FFL and service-provider handlers.

Among newer systems, Temporal [Tem] and Cadence [Cad] are the prominent representatives of the workflow-as-code school descended from Microsoft's AMBROSIA [GKR+19] and Uber's internal Cherami. Their core contribution is deterministic replay of general-purpose code against a recorded event history, a powerful mechanism with (I will argue in Chapter 9) specific, structural costs.

3.3 Distributed job processing

The Celery family (Celery [Sol21], Sidekiq [Per17], Resque) popularised the broker-backed task queue for web-application background jobs. These systems optimise for short-lived tasks, many per second, with minimal coordination semantics beyond retry and acknowledgement. Facetwork differs in that it supports long-running, multi-stage workflows with typed outputs, but it shares the fundamentally decentralised pull model.

Kubernetes Jobs and CronJobs [Kub] are a distinct point in the design space: work is described as pod specifications, coordination is the Kubernetes control plane, recovery is pod-level restart. Argo Workflows [Arg] layers a workflow abstraction on top. The combined system works well when the workflow is a DAG of container executions and poorly when the workflow is one long-lived process with internal stages, which is exactly Facetwork's target workload.

3.4 Live updatability

The idea that a running distributed system should be updatable without full restart has been pursued by the Erlang/OTP community since the 1980s [Arm03], where "hot code loading" is a first-class feature. Industrial systems have mostly given this up in favour of blue/green deployments and rolling restarts. Facetwork's live handler registration is not as radical as Erlang's module swap — the handler code itself still runs in a Python process that can be restarted if needed — but the registration-in-database model means that *which handler runs* can be changed without restarting anything. This turns out to be sufficient for most operational needs.

The Kubernetes ecosystem has normalised the expectation that pods come and go during a running workflow, but the application code running *inside* each pod is fixed for the pod's lifetime. Facetwork's model pushes updatability down one level: inside a single long-running runner, the set of handlers loaded can change, the per-task timeout regime can change, and the server's operational state (running, quarantine, shutdown) can change. This is closer to the Erlang model than to the Kubernetes one.

Part II — The Facetwork Design

Chapter 4. FFL: A Flow Algebra

4.1 Design goals

The Facetwork Flow Language is designed with four goals in mind:

1. **Authorable by domain experts.** An epidemiologist describing a genomic analysis pipeline, an urban planner describing a geocoding workflow, or an operations engineer describing a deployment cadence should be able to write and read FFL without being a Python programmer. This constrains the syntax to remain shallow.
2. **Statically checkable.** The parser and type checker should catch as many topology and typing errors as possible before execution. This implies a typed grammar with compile-time resolution of facet references.
3. **Implementation-agnostic.** FFL describes *what* each facet should do, typed by its parameters and return clause, but does not describe *how* it is implemented. A facet is implemented by a handler, and the handler is picked up by the runtime at execution time. The same FFL declaration can bind to a Python handler, a Scala handler, a remote service handler, or — in a special case — a language-model prompt.
4. **Composable.** Sub-workflows, mixins, implicit facets, and parallel branches all compose without tricky operator precedence. A reader should be able to trace the data flow with a finger.

4.2 Core constructs

An FFL program consists of **namespace declarations** containing **facet** and **schema** declarations. A facet is a typed computational unit; a schema is a typed record. An **event facet** (prefixed with `event`) is one whose computation is performed by an external handler rather than inline. A **workflow** is a facet designated as an entry point for execution.

A facet has a name, a parameter list, and either an inline body or a return clause. In the inline case, the facet is derived entirely from its parameters. In the event-facet case, the implementation is delegated to a handler registered by name. In the workflow case, the facet is marked as executable and triggers the runtime to create a `fw:execute:<WorkflowName>` task.

```
namespace osm.ops {

  schema OSMCache {
    url: String
    path: String
    date: String
    size: Int
    wasInCache: Bool
  }

  event facet PostGisImport(cache: OSMCache, region: String, force: Bool)
    => (output: OSMCache)
```

```
  workflow ImportRegion(region: String, force: Bool) => (output: OSMCache) andThen {
    c = DownloadPBF(region = $.region)
    s = PostGisImport(cache = c, region = $.region, force = $.force)
    yield ImportRegion(output = s.output)
  }

}
```

A workflow's signature is `=> (name: Type, ...)` declaring its typed return fields, followed by an `andThen { ... }` block that is the workflow's body. Inside that body, parameters of the enclosing workflow are reached through the **container-attribute syntax** `$.name`; locally-defined assignments (`c`, `s`) are reached by bare name. The `$` distinguishes *reaching out of the current block into its container* from *reaching across statements within the current block*, and the distinction is enforced by the compiler. This is the simple case. The power of FFL comes in composition.

4.3 Composition: `andThen`, `foreach`, and `when`

`andThen { ... }` introduces a **block** — a lexical scope containing a set of assignments. Two rules govern blocks:

1. **Statements within a block share a scope.** Any assignment in the block may reference any other assignment in the *same* block by name, and the runtime orders execution according to the resulting data-dependency partial order.
2. **Statements cannot reach across block boundaries.** An assignment in one block cannot name an assignment in a sibling block or an outer block. To reference values from the enclosing workflow or facet — its parameters, its declared return fields — the block uses the container-attribute syntax `$.name`.

Sibling blocks — two `andThen { ... }` blocks attached to the same container, or two `andThen foreach` expansions in the same scope — execute concurrently with each other. Consequently, a workflow that wants analysis to happen *after* extraction cannot place the two in separate sibling blocks; it must either place them in the *same* block (letting data flow order the work) or chain the blocks so that the later block sees the earlier block's outputs through the container scope.

The single-block form is usually what you want:

```
schema RouteStats    { count: Int, total_length_km: Float }
schema AmenityStats  { count: Int, by_type: Map[String, Int] }
schema BuildingStats { count: Int, total_area_m2: Float }

event facet ExtractRoutes(source: OSMCache)       => (result: RouteFeatures)
event facet ExtractAmenities(source: OSMCache)    => (result: AmenityFeatures)
event facet ExtractBuildings(source: OSMCache)    => (result: BuildingFeatures)
event facet RouteStatistics(routes: RouteFeatures)        => (result: RouteStats)
event facet AmenityStatistics(amenities: AmenityFeatures) => (result: AmenityStats)
event facet BuildingStatistics(buildings: BuildingFeatures) => (result: BuildingStats)

workflow ImportAndAnalyse(region: String)
    => (routes: RouteStats, amenities: AmenityStats, buildings: BuildingStats)
    andThen {
  c             = DownloadPBF(region = $.region)
  s             = PostGisImport(cache = c, region = $.region)
```

```
  routes        = ExtractRoutes(source = s.output)
  amenities     = ExtractAmenities(source = s.output)
  buildings     = ExtractBuildings(source = s.output)
  routeStats    = RouteStatistics(routes = routes.result)
  amenityStats  = AmenityStatistics(amenities = amenities.result)
  buildingStats = BuildingStatistics(buildings = buildings.result)
  yield ImportAndAnalyse(
    routes    = routeStats.result,
    amenities = amenityStats.result,
    buildings = buildingStats.result,
  )
}
```

Two features of this example deserve explicit commentary, because they are easy for new FFL authors to get wrong.

First, a step is not a value. `routes`, `amenities`, `buildings` — these names bind to *steps*, not to their outputs. A step is a node in the execution graph; it has a lifecycle (pending, running, completed, errored) and, on completion, exposes its declared return fields as named attributes. You therefore cannot `yield routes`; you can only yield the *attributes* of a step. Every event facet declares its return as `=> (name: Type, ...)`, and downstream code reads a completed step's values via `stepName.fieldName`. In this example, the extraction facets declare `=> (result: ...Features)`, the statistics facets declare `=> (result: ...Stats)`, and the `PostGisImport` facet (from §4.2) declared `=> (output: OSMCache)` — so we see the importer's output as `s.output` and each statistic as `routeStats.result`, `amenityStats.result`, `buildingStats.result`.

Second, the workflow is its own facet. A workflow's return clause declares the fields a caller of the workflow sees, and the `yield` statement is a constructor-like call on the workflow's own name assigning values to those fields. Here the workflow declares three return fields (`routes`, `amenities`, `buildings`), each with its own schema, and the `yield ImportAndAnalyse(...)` populates them from the completed statistics steps. A consumer calling `ImportAndAnalyse` from another workflow would then see a value with `.routes`, `.amenities`, `.buildings` — all three statistic schemas available in parallel — and could reach into any of them.

With those two rules in hand, the execution plan follows from data flow: `c` runs first; `s` runs after `c`; the three `Extract*` facets run concurrently once `s` completes; each `*Statistics` step runs after its corresponding extraction; and the workflow yields once all three statistics steps have produced their `result` attributes. The source order of the eight assignments has no bearing on the schedule; reordering them would produce an identical plan.

If the same workflow *were* written with two sibling `andThen` blocks, it would be broken, because the second block cannot see `routes` or `amenities`:

```
// INCORRECT — do not do this
workflow BadImportAndAnalyse(region: String) => (stats: RouteStats) andThen {
  c = DownloadPBF(region = $.region)
  s = PostGisImport(cache = c, region = $.region)
} andThen {
  // ERROR: `s`, `routes`, `amenities` are not in scope here;
  // only $.region (a container attribute) is visible.
  ...
}
```

The two `andThen` blocks run concurrently; the second block has no access to names defined in the first. The only way for the second block to observe an upstream result is if that result has been promoted to a container attribute — a pattern that a few workflows use but most avoid in favour of the single-block form.

4.3.1 When to use several sibling `andThen` blocks

If statements cannot cross block boundaries, and if a single block is already a data-flow-ordered concurrent computation, one might ask what the *point* of multiple sibling `andThen` blocks is. The answer is **independence and readability**.

Consider a workflow whose inputs split naturally into several unrelated processing tracks — for example, one that ingests a PBF file on one track and a CSV file on another, each producing a different piece of the final result. One could cram all of the statements into a single block and rely on the data-flow partial order to separate the tracks. For a handful of statements this is fine. For more, it becomes a wall of assignments whose independence is only discoverable by tracing variable names across lines. A reader who wants to understand the CSV branch has no syntactic signal of where it begins and ends; it is intermixed textually with the PBF branch even though the two never interact.

Multiple sibling `andThen` blocks let the author *show* that independence. Each block is its own scope, each runs concurrently with its siblings, and — this is the key — **each block ends in its own `yield`, populating a different field of the workflow's declared return**:

```
workflow SummariseRegion(region: String, census_csv: String)
    => (geo: GeoSummary, demo: DemoSummary)
    andThen {
  c     = DownloadPBF(region = $.region)
  s     = PostGisImport(cache = c, region = $.region)
  geo   = GeoSummariser(source = s.output)
  yield SummariseRegion(geo = geo.result)
} andThen {
  rows  = LoadCensus(path = $.census_csv)
  stats = DemographicStats(rows = rows.result)
  yield SummariseRegion(demo = stats.result)
}
```

The two blocks share only the workflow's container attributes (`$.region`, `$.census_csv`) and the workflow's declared return fields (`geo`, `demo`). They run at the same time — `DownloadPBF` and `LoadCensus` are both emitted as soon as the workflow starts — and each block independently contributes one field of the final return by yielding into it. When both blocks have yielded, the workflow is complete and its return is `{ geo: ..., demo: ... }` populated from both tracks.

The contrast with a single-block equivalent is instructive:

```
// Works, but harder to read as the workflow grows:
workflow SummariseRegion(region: String, census_csv: String)
    => (geo: GeoSummary, demo: DemoSummary)
    andThen {
  c     = DownloadPBF(region = $.region)
  s     = PostGisImport(cache = c, region = $.region)
  geo   = GeoSummariser(source = s.output)
  rows  = LoadCensus(path = $.census_csv)
  stats = DemographicStats(rows = rows.result)
```

```
  yield SummariseRegion(geo = geo.result, demo = stats.result)
}
```

Both forms produce the same execution plan. The multi-block form makes the structural independence of the two processing tracks visible at the top level of the workflow, and it localises each track's internal names (`c`, `s`, `geo` vs. `rows`, `stats`) to the scope where they matter. The single-block form mixes them.

A rule of thumb: **use separate `andThen` blocks when separate output fields are produced by wholly independent work**. Use a single block when there is real data flow tying the statements together. Blocks, then, are a notation not only for scope but for intent — the author tells the reader, by reaching for a second `andThen`, that this is a distinct track of work whose only interaction with its siblings is through the workflow's return.

One caveat: each `yield` assigns only the fields it names. A workflow that declares `=> (geo: GeoSummary, demo: DemoSummary)` and has two sibling blocks, one yielding `geo` and the other yielding `demo`, is complete only when both blocks have yielded. A yield that omits a declared field does not clear it; a field that is never yielded remains unset and the workflow does not complete. The compiler verifies that, across all reachable `yield` statements, every declared return field has at least one assignment.

`andThen foreach` iterates over a collection, creating one parallel branch per element, with the iteration variable exposed through the container:

```
workflow ImportAllCountries(regions: [String]) andThen foreach region in $.regions {
  r = ImportRegion(region = $.region, force = false)
}
```

The runtime creates one child execution per region; they execute concurrently, subject to the fleet's concurrency budget.

`andThen when` is the conditional dispatch:

```
result = s andThen when {
  case s.size > 1000000000 => { LargeRegionAnalysis(data = s) }
  case s.size > 100000000  => { MediumRegionAnalysis(data = s) }
  case _                   => { SmallRegionAnalysis(data = s) }
}
```

The `andThen when` expression evaluates exactly one branch. Together, `andThen`, `andThen foreach`, and `andThen when` provide enough control flow for all the workflows we have encountered without needing general-purpose loops. The absence of unbounded loops is not an accident; it is a deliberate restriction that makes topology statically tractable. Bounded iteration (foreach) is sufficient because the collections it iterates over are themselves either the result of upstream facets (and thus have known size at the iteration point) or parameters to the workflow.

4.4 Data-dependency semantics: concurrency by default

A defining property of FFL, and one that distinguishes it sharply from imperative workflow-as-code, is that **the textual order of statements in an `andThen` block does not determine execution order**. Execution order is determined solely by data dependencies inferred from the expressions.

Consider:

```
workflow CompareStates() => (result: ComparisonReport) andThen {
  ca = Download(input = "ca.pdf")
  wa = Download(input = "wa.pdf")
  comparison = Compare(input1 = ca.output, input2 = wa.output)
  yield CompareStates(result = comparison.report)
}
```

The two `Download` facets have no data dependency on each other. FFL's semantics are that they execute *concurrently*: the workflow evaluator emits two tasks immediately, either of which may be claimed and executed by a different runner on a different host. The `Compare` facet references the outputs of both downloads; it is blocked until both have produced their typed results. When both have completed, the `Compare` task is emitted, claimed by whichever runner is next to poll, and executed.

The important negative claim is that the source order `ca; wa; comparison` does not *cause* `ca` to run before `wa`. Swapping the first two lines would not change the execution plan. A reader wishing to understand the runtime shape of a workflow should trace its data flow, not its statement order — and, in an FFL program, the data flow is written directly in the expressions.

This semantic is the same discipline that pure functional languages adopt with respect to let-binding order, and it is adopted here for the same reason: it frees the runtime to schedule work as aggressively as the data constraints permit, without the author having to reason about which statements they should write in which order. Imperative workflow-as-code systems approximate this through framework-specific constructs — a Temporal workflow that wants two activities to run in parallel must say so with something like `futures = [activity_a.execute_async(), activity_b.execute_async()]; workflow.wait_for_all(futures)`, and Airflow's TaskFlow API must be nudged with explicit `.expand()` or DAG-level parallelism settings. FFL needs no such ceremony because concurrency is the default and *sequencing* is what requires an explicit data dependency.

The `yield` at the workflow's tail declares the return value. The runtime computes the transitive closure of that value's dependencies and schedules them with maximal parallelism; anything not reachable from a yielded value is, semantically, unused and will typically be flagged as a compile-time warning. For authors who want explicit sequencing between steps that do not share a natural data dependency — for example, two operations with ordering requirements via external side effects — FFL offers successive `andThen` blocks rather than relying on source order. This makes the dependency visible: if a step must precede another, the source says so by placing them in successive `andThen` blocks rather than in the same block.

A single `andThen` block, then, is best understood as an **unordered set of assignments with a data-flow partial order** rather than a sequence of statements. The runtime schedules the set respecting that partial order; the reader reasons about the set respecting that partial order; the compiler verifies the partial order is well-formed.

4.4.1 Yield semantics: immutable during execution, aggregated on block completion

FFL's block-mediated coordination (§5.8) has a corresponding data-model discipline worth making explicit. **During the execution of a block's steps, the block's input and output attributes are immutable.** Every step in the block sees the same snapshot of container attributes (`$.region`, `$.census_csv`, and so on) and the same bindings to its siblings; nothing a step does while running can change what another step in the same block observes. There is no mid-execution mutation for an author to reason about, and therefore no data race for the implementation to defend against.

Yields do not mutate; they accumulate. A `yield` statement does not write into the container's output fields the moment it is executed. Instead, yields are **collected** as the block's steps complete, and when the block as a whole is complete, the accumulated yields are **applied to the container's declared return fields as a single atomic assignment**. The model is closer to a transactional commit at the block boundary than to imperative assignment: no partially-yielded state is ever visible to another block, another step, or the workflow evaluator.

This discipline is what makes sibling `andThen` blocks safe to run in parallel without any coordination between them. Two blocks that yield into different return fields of the same workflow cannot race, because neither yield is visible until its own block has finished; and the block evaluator of §5.8 processes the aggregated yields serially as each block completes.

Collection and map return fields aggregate across multiple yields. If a workflow declares a return field whose type is `[T]` or `Map[K, V]`, and the field receives more than one yield — across sibling blocks, across iterations of `andThen foreach`, or across branches of an `andThen when` that each contribute to it — the yields are merged rather than overwriting each other. For `[T]`, the contributions are appended; for `Map[K, V]`, the contributions are unioned, with later keys taking precedence over earlier ones. This is the semantic that makes `andThen foreach` useful as an aggregate reducer: each iteration's yield adds to the collection, and the final field is the merged result of all iterations.

A worked example:

```
workflow AllRegionStats(regions: [String])
    => (stats: [RegionStats])
    andThen foreach region in $.regions {
  s = ImportRegion(region = $.region, force = false)
  r = RegionStatistics(source = s.output)
  yield AllRegionStats(stats = [r.result])
}
```

Each iteration yields a single-element list into `stats`. The workflow's final `stats` is the concatenation of every iteration's yield. Scalar return fields do not aggregate in this way — a workflow that yields a scalar twice is a compile-time error, because there is no ambiguity-free merge of two scalars of unrelated provenance. Aggregation is opt-in through the return field's declared type.

The practical effect of these rules is that `yield`, viewed from inside the language, behaves like the commit phase of a transaction scoped to a block, and viewed from outside the language, behaves like a reducer that absorbs contributions from every path that reaches it. Both views are correct; neither requires the author to reason about interleaved concurrent writes.

4.4.2 Pass-by-step: parameters typed as facets

The reference forms presented so far carry a single value — `s1.field` selects one attribute, `$.field` reads one of the enclosing facet's bound inputs. FFL admits a third form: when a parameter's declared type is itself a facet name, the argument bound to that parameter is **a reference to a whole step** rather than a copy of one of its fields.

```
facet Value(input: String) => (output: String)

facet Consumer(facetRef: Value) => (output: String) andThen {
    yield Consumer(output = $.facetRef.output)
}

workflow Demo(input: String) => (output: String) andThen {
    s1 = Value(input = $.input)
    s2 = Consumer(facetRef = s1)
    yield Demo(output = s2.output)
}
```

`facetRef: Value` declares that `Consumer`'s `facetRef` parameter expects a step whose call target is the facet `Value`. The argument is the bare step name `s1`, not `s1.output`. Inside the consumer body — and inside its handler, if it is an event facet — `facetRef` is a **FacetRef**: a small immutable record carrying the upstream step's identity (`step_id`, `workflow_id`, `facet_name`). The reference's contents are resolved lazily: `$.facetRef.output` causes the evaluator to load the upstream step's record on first use and read the named attribute from it; subsequent reads in the same evaluation context are served from a per-tick cache. Both bound input attributes and computed output attributes are accessible through this path, because the dependency tracker blocks the consumer's task from being claimed until the upstream step is in a terminal completed state with both kinds of attribute persisted.

The motivation is the same as the motivation for any reference type: avoiding the need for the producer to project every potentially-interesting field into the signature in advance. A pipeline that hands a whole `OSMCache` step downstream can let each consumer read just the fields it needs, without the producer enumerating them. The cost is paid in the consumer's own evaluation, not at the call site, and the reference is **read-only by construction** — there is no write API on the FacetRef and no way for a consumer to mutate the upstream step. A handler that wishes to derive a new value from a referenced step does so by returning its own outputs, which become the handler's own step record, distinct from the upstream's.

The validator enforces exact facet-name match (rule `STEP_REF_FACET_MISMATCH`): passing a step of a different facet to a FacetRef parameter is a compile-time error. Mixin compatibility — accepting a step whose facet *includes* the declared type via composition — is intentionally not considered in the first cut and is left as an extension point.

This third reference form composes naturally with the data-dependency discipline introduced above. A FacetRef is, for the purposes of dependency analysis, a reference to its target step exactly as `s1.field` would be; the dependency tracker reads `path[0]` regardless of whether the path is bare or dotted. Concurrency-by-default and block-scoped immutability of inputs (§4.4) hold without modification. The new form simply extends the kinds of value an `andThen` block can pass around, without altering the rules by which the runtime decides what runs when.

A facet signature may declare mixins with the syntax `with M() as <alias>`, and a consumer that holds a FacetRef can then address each aliased mixin's sub-step through `$.fref.<alias>.<field>`. This makes the per-mixin work visible to downstream consumers without forcing the producing facet to re-project mixin outputs into its own returns. The alias shares the consumer-side namespace with the facet's own params and returns; a compile-time rule rejects collisions — for instance, `facet F3(m1: String) with M1() as m1` is illegal because the alias `m1` shadows a primary parameter. Mixins declared without an `as` alias are deliberately unreachable from FacetRef consumers: they remain part of the facet's composition but are private to it, the namespace-discipline analogue of an unexported method. Aliases therefore serve two purposes at once — they give consumers a stable name to address mixin sub-steps, and they form an opt-in surface boundary that determines what a facet exposes through pass-by-step.

The same `with` keyword that attaches mixins to a signature also chains additional targets in a `yield` statement, so the producing facet's body reads symmetrically with its declaration:

```
facet F2(input: String) => (output: String)
    with M1() as m1
    with M2() as m2
    andThen {
        yield F2(output = $.input)
            with M1(output = $.input)
            with M2(output = $.input)
    }
```

A single `yield F2(...) with M1(...) with M2(...)` is parsed as one statement per target — the compiler unpacks the chain into independent yields, each subject to the usual yield-target rules. The chained form removes the visual asymmetry between a signature that lists its mixins on stacked lines and a body that would otherwise need three separate `yield` statements to fan results back across them.

The mixin sub-step a consumer sees through `$.fref.<alias>` is, in the current design, a **view** over the parent's persisted yields rather than a separately-executed step. When a consumer reads `$.f2.m1.output`, the runtime resolves it by examining `f2`'s yield steps, selecting those whose target facet is the one bound to `m1` on F2's signature, and exposing the merged contributions as the mixin sub-step's `returns`. This sits naturally inside the yield-aggregation discipline of §4.4.1: a FacetRef alias gives a name to a slice of what `MixinCaptureBegin` would have folded into the parent's returns, indexed by the producing mixin rather than collapsed into a single namespace. The consumer never observes mixin execution as a distinct activity; it observes the durable record the producer wrote on its way to completion. If a future revision adds first-class facet-signature mixin execution — separate sub-steps with their own state, retries, and dashboard visibility — the view becomes a direct lookup of those persisted rows, and no consumer-side code changes.

4.5 Mixins, implicits, and inheritance-by-composition

Mixins let one facet be defined as the combination of several others:

```
event facet EnrichedImport(cache: OSMCache) => (result: OSMCache)
  with PostGisImport(cache = $.cache, region = $.cache.region)
```

```
  with IndexSpatial(source = $.cache)
  with PublishAudit(source = $.cache)
```

Mixins are not inheritance in the object-oriented sense — FFL has no classes — but they serve a similar purpose: factoring shared sub-computations out of many specific ones.

4.5.1 Execution phases: mixins, primary facet, step-level blocks

The full execution model for a call with mixins, a primary-facet body, and a step-level `andThen` is a layered, phase-by-phase evaluation. It is easiest to see in a worked example with deliberately small names.

```
event facet LoadA(key: String) => (a: Int)
event facet LoadB(key: String) => (b: Int)
event facet LoadC(key: String) => (c: Int)
event facet Sum(a: Int, b: Int) => (total: Int)
event facet Label(a: Int, b: Int, tag: String) => (text: String)
event facet Format(a: Int, b: Int, c: Int, doubled: Int, tagged: String)
    => (result: String)

// A composite facet:
//   - two mixins (LoadA, LoadB), each contributing one output field
//   - two primary andThen blocks, each contributing another output field
facet Gather(key: String)
    => (a: Int, b: Int, doubled: Int, tagged: String)
    with LoadA(key = $.key)
    with LoadB(key = $.key)
    andThen {
      s = Sum(a = $.a, b = $.b)
      yield Gather(doubled = s.total * 2)
    } andThen {
      m = Label(a = $.a, b = $.b, tag = $.key)
      yield Gather(tagged = m.text)
    }

// A workflow that uses Gather as a step and post-processes with a
// step-level andThen attached to the call site.
workflow Report(key: String) => (message: String) andThen {
  g = Gather(key = $.key) andThen {
    c = LoadC(key = $.key)
    f = Format(a = g.a, b = g.b, c = c.c, doubled = g.doubled, tagged = g.tagged)
    yield Report(message = f.result)
  }
}
```

When `g = Gather(...)` is evaluated, the runtime proceeds in three distinct phases:

Phase 1 — Mixin phase. The two mixins declared by `with LoadA(...)` and `with LoadB(...)` execute **concurrently**. Each mixin sees only `Gather`'s *input* attributes (`$.key`) and contributes to a different output field of `Gather` (`a` from `LoadA`, `b` from `LoadB`). The mixin phase completes only when every `with` clause has yielded. At the boundary between Phase 1 and Phase 2, the output fields yielded by mixins — here, `a` and `b` — are atomically applied to `Gather`'s container attributes.

Phase 2 — Primary facet phase. The primary facet's `andThen` blocks are now eligible to run. Crucially, at this point `$.a` and `$.b` *are* readable from inside Phase 2, because the mixin phase has committed them to the container. The two primary `andThen` blocks execute concurrently with each other (they are sibling blocks per §4.3), each reading `$.a` and `$.b` and contributing a different output field (`doubled` and `tagged`). Neither primary block can read the other's yields — scalar outputs yielded in sibling blocks do not become visible to each other mid-phase — but both can freely read mixin outputs. When both primary blocks complete, their yields are atomically applied to the container, so that by the end of Phase 2, `Gather`'s four output fields (`a`, `b`, `doubled`, `tagged`) are all populated.

Phase 3 — Step-level block (at the caller's site). The `andThen { ... }` appended to the call `g = Gather(key = $.key)` is the *step-level* block. It executes after Phases 1 and 2 have fully completed, which means it sees the fully-assembled step output through the step binding `g`: `g.a`, `g.b`, `g.doubled`, `g.tagged`. In this example it also performs a fresh `LoadC` call and then calls `Format` using all of the step's outputs plus the newly-loaded `c`. The step-level block belongs lexically to the *caller* (`Report`), not to `Gather`, so it can read the caller's own container attributes (`$.key`) and any earlier bindings in the caller's block.

Three visibility rules follow, which taken together capture the whole model:

1. **Phase 1 sees only inputs.** Mixins cannot see the primary facet's yields or other mixins' yields. They are pure functions of the container's inputs.
2. **Phase 2 sees inputs plus Phase-1 yields.** Primary `andThen` blocks can read every attribute yielded by any mixin, because Phase 1 has already committed. They cannot read sibling primary-block yields — those are still accumulating — and they cannot read attributes that will only be produced by the step-level block.
3. **Phase 3 sees the completed step.** The step-level block at the caller's site reads the step's attributes as `stepName.fieldName`, and only runs after every field has been assigned by Phase 1 or Phase 2.

The model composes cleanly with everything said earlier. The atomicity described in §4.4.1 — "yields are collected and applied atomically at block completion" — now has a richer story: it applies at every phase boundary, not just at the end of a single block. Phase 1's aggregated yields are committed atomically to the container; Phase 2's aggregated yields are committed atomically; Phase 3 sees the committed total. No phase ever observes a half-populated step, and no concurrent writers within a phase can race because the phase's yields are accumulated and applied only at its boundary.

This is the whole of the composition model, stated as plainly as the author can manage. A reader who has absorbed it can then read arbitrarily complex FFL with confidence: mixins are the "before" contributions, primary `andThen` blocks are the "during" contributions, step-level `andThen` is the "after" block at the caller's site, and every transition between phases is an atomic commit of the phase's accumulated yields.

Implicit facets are parameters resolved by name rather than by position:

```
workflow ImportWithImplicitLogger(region: String) => (result: OSMCache) andThen {
  implicit logger = AuditLogger(region = $.region)
  c = DownloadPBF(region = $.region)
  s = PostGisImport(cache = c, region = $.region)
```

```
  // The logger is implicitly passed to any facet that declares it
  yield ImportWithImplicitLogger(result = s.output)
}
```

Implicit facets dramatically reduce the parameter-threading overhead of workflows with shared concerns (logging, metrics, authentication). They are resolved statically at compile time: the compiler matches implicit declarations to facet parameters by name and type, erroring if more than one candidate or none match. This gives implicit resolution the static checking that runtime service-locator patterns lack.

4.6 Error handling: `catch` and `catch when`

FFL makes error handling part of the flow language rather than relegating it to handler internals:

```
s = DownloadPBF(region = $.region) catch when {
  case err.type == "NetworkError" => {
    retried = DownloadPBF(region = $.region, mirror = "backup")
  }
  case err.type == "DiskFullError" => {
    cleared = CleanCache()
    retried = DownloadPBF(region = $.region)
  }
  case _ => { FailWorkflow(reason = err.message) }
}
```

`catch when` cases are statically matched against a typed error schema; the compiler verifies that every case is reachable and that the final catch-all is present if the cases are not exhaustive. This lifts error handling from a handler-internal `try/except` — invisible to the orchestrator — into a topology-visible construct that the dashboard renders and that the compiler can type-check.

4.7 Prompt blocks: first-class LLM integration

A prompt block is a facet whose implementation is an LLM invocation:

```
event facet TriageIncident(logs: [String]) => (report: TriageReport) {
  prompt {
    system "You are an experienced site reliability engineer."
    template "Given these log lines, identify the probable root cause: {{$.logs}}"
    model "claude-opus-4-6"
  }
}
```

Prompt blocks are event facets whose handler is the Facetwork runtime itself rather than user code. The runtime handles the API call, retries, token accounting, and response parsing against the declared return schema. This is a concrete example of FFL's decoupling of *what* from *how*: the prompt facet declares a typed input and typed output, and the implementation is supplied by the runtime via a generic LLM-call handler.

One subtler point worth noting: the introducer (`prompt`) and the three directive words (`system`, `template`, `model`) are deliberately **soft keywords** — recognised only in the prompt-block position via a contextual lexer plus a semantic check in the AST builder. The same identifiers remain available everywhere else in the grammar: as parameter names (`event facet CreateMessage(prompt: String, system: String, model: String)`), as schema field names, as named-argument keys. This matters once a package like the Anthropic SDK wrapper, where `prompt` and `system` are the natural names for the request payload, becomes part of the ecosystem: hard-reserving those words would force every such package to use awkward synonyms (`user_prompt`, `system_prompt`). Soft keywords are also a small concession to AI-generated FFL, where the surface vocabulary an LLM reaches for overlaps almost exactly with the prompt-block lexicon.

4.8 Script blocks: controlled escape hatch

Occasionally a pure-FFL description is not enough, and the workflow author genuinely needs to compute something:

```
s = ComputeDerivedSize {
  script python """
    return {"size": cache.size * multiplier}
  """
}
```

Script blocks are executed in a sandboxed Python subprocess with a controlled input/output boundary. They are the escape hatch of last resort and the compiler warns on their use. Most FFL programs contain none; the ones that contain many are a code smell indicating that too much logic is escaping the language.

4.9 Type system

FFL has a Hindley-Milner-flavoured type system specialised to workflow concepts. The primitive types are `String`, `Int`, `Float`, `Bool`, `Null`. Compound types include `List[T]`, `Map[K, V]`, and record types declared via `schema`. There is no parametric polymorphism on user-defined facets — every facet has a concrete type signature — but schemas can be recursive, and the type checker handles structural subtyping on record literals, so that a value of type `{ name: String, size: Int, extra: String }` can be passed where `{ name: String, size: Int }` is expected.

The type checker runs before emission. Its job is to verify that every facet call's parameters match its declared signature, that implicit resolution is unambiguous, that `yield` values are compatible with the workflow's declared return, and that `catch when` cases exhaust the error type. Compilation errors include line and column numbers, so that error messages can be rendered in the dashboard as clickable anchors in the FFL source.

4.10 Why a DSL, not a library?

The natural question is why Facetwork needs a new language at all. Why not simply expose the facet/workflow abstractions as a Python library with decorators? This is the Temporal/Prefect/Dagster approach. I give three reasons why the DSL choice is not merely aesthetic but functional.

First, compile-time topology checking. In a library-based system, the workflow graph is constructed by running Python code. Static analysers can be written that do some checking (Dagster's linter, Temporal's workflow-sandbox), but they are always playing catch-up with what the dynamic language actually allows. In FFL, the graph is the parse tree. Type errors, unreachable branches, and misnamed facet references are caught by a parser with <1 KLOC of Lark grammar.

Second, compile-time separation of concerns. The author of a workflow should not need to know how any particular facet is implemented. In a library-based system, the workflow code imports the step implementations directly; refactoring the implementation affects the workflow author. In FFL, the workflow author references facets by qualified name; the binding to a handler happens at registration time. An operator can swap the implementation of `osm.Source.PostGIS.ExtractRoutes` without touching any workflow.

Third, IDE and dashboard tooling. FFL's small, fixed grammar makes it feasible to build language servers, syntax-aware dashboards, and visual editors in a way that general-purpose code does not. The Facetwork dashboard renders FFL source inline with execution state, linking each step's current status to the corresponding source line. An equivalent affordance for Python workflow code would require running a debugger.

The cost of a DSL is real — users must learn it, documentation must be maintained, the compiler must be kept correct — but the benefits accrue monotonically as workflows grow in number and authors multiply. In the limit of one workflow written by one author, there is no reason to use a DSL. In the limit of fifty workflows written by ten authors across five domains, there is no reason not to.

Chapter 5. The Distributed Coordination Model

5.1 Overview

Facetwork's coordination model has four moving parts:

1. **Tasks**: rows in a MongoDB collection representing pending or in-flight work.
2. **Runners**: long-running server processes that claim tasks.
3. **Handlers**: functions within a runner that execute a specific facet.
4. **The workflow evaluator**: a component within each runner that, on completion of a step, updates the workflow graph and enqueues the next tasks.

The central design property is: **there is no coordinator**. No master scheduler, no message broker, no lock service. Every runner is symmetrical; the database is the only shared state; coordination decisions are made by atomic database operations.

5.2 The task document

Each task is one MongoDB document. The relevant fields are:

```
uuid: str                    # task identifier
name: str                    # facet name to dispatch
runner_id: str               # workflow instance id
workflow_id: str             # workflow definition id
flow_id: str                 # sub-flow id within the workflow
```

```
step_id: str                    # execution step id
state: str                      # "pending" | "running" | "completed" | ...
created, updated: int (ms)
lease_expires: int (ms)         # lease held by the claimer
task_heartbeat: int (ms)        # last heartbeat from the handler
server_id: str                  # current claimer's server uuid
retry_count, max_retries: int
timeout_ms: int                 # per-handler timeout (0 = use default)
stage_budget_expires: int (ms) # per-stage timeout extension (Chapter 8)
stage_name: str                 # name of active stage, for diagnostics
data: dict                      # serialised handler input
error: dict | None              # set on failure
```

The invariant we maintain is: **for every `(step_id, state="running")` pair, there is at most one task document**. This invariant is enforced by a *partial unique index* on that pair. MongoDB's partial index feature means the uniqueness applies only to documents matching the filter (`state = "running"`), which is exactly what we want: many tasks may be pending, but only one may be running per step at any time.

5.3 The claim protocol

A runner claims work by issuing a `find_one_and_update`:

```
doc = tasks.find_one_and_update(
  {
    "state": "pending",
    "name": {"$in": eligible_task_names},
    "task_list_name": self.task_list,
    "$or": [
      {"next_retry_after": {"$exists": False}},
      {"next_retry_after": 0},
      {"next_retry_after": {"$lte": now}},
    ],
  },
  {"$set": {
    "state": "running",
    "updated": now,
    "lease_expires": now + lease_ms,
    "server_id": self.server_id,
  }},
  return_document=ReturnDocument.AFTER,
)
```

The update is atomic at the document level. MongoDB's single-document operations are linearisable under the default write concern (majority), meaning that no two runners can both see the document as pending and both claim it. The partial unique index provides a second-level defence: if two concurrent `find_one_and_update` calls somehow raced past the atomic selection — they will not, but defence in depth is cheap — the unique index would reject the duplicate `running` document.

This is the entire claim protocol. There is no broker. There is no coordinator. There is no leader election. There is not even a task distribution algorithm in the usual sense: each runner asks the database for a task and is given one or not.

5.4 Per-runner fetch and workload partitioning

The claim protocol of §5.3 describes a single atomic transition. The fleet-wide behaviour follows from each runner executing that transition independently. There is no central dispatcher, no work-stealing queue, and no inter-runner channel: each runner's poll loop is a closed feedback circuit between itself and the database.

Polling is local. Every runner runs its own poll loop, with its own period (configurable; default one second). RunnerA's poll cadence has no effect on RunnerB's; a runner that is busy executing a long task is the only thread blocked, and other runners continue claiming and dispatching normally. There is no shared in-memory data structure to contend on between processes — even processes on the same physical machine coordinate only through the database.

Eligibility is a query, not a routing decision. A runner's claim issues the find_one_and_update with three filter components:

1. `name ∈ <handler-set> ∪ {fw:execute, fw:resume}` — only tasks for facets this runner can dispatch, plus the protocol tasks every runner participates in.
2. `task_list_name = <runner's configured list>` — a single string identifier; default `"default"`.
3. `state = "pending" ∧ retry_eligible` — claimable in the lifecycle sense.

The handler-set filter is built once at runner startup. A registry-mode runner enumerates the `handler_registrations` collection (§6.1), verifies that each entry's module is importable in *this* process, and registers a proxy only for those that load. In a per-example deployment — one container per `fwh_*` package — each container's handler set is exactly that package's facets, and the name filter alone produces deterministic routing: no other runner's filter can match the same task.

Two regimes follow. When two runners' filters intersect (both can dispatch the same name on the same list), a claim races between them; MongoDB's per-document atomicity ensures exactly one wins and the other returns `None` and polls again. No fairness is guaranteed across the racers; whichever finishes the database round-trip first claims. When two runners' filters are disjoint, their queries return tasks from non-overlapping subsets of the collection and they cannot race at all. The expected throughput of a runner is independent of the load on a runner whose filter does not intersect — a thousand-task backlog on an `osm` runner adds zero latency to claims on an `anthropic` runner, because the latter's query never sees those documents.

Workload partitioning via `task_list_name`. The `task_list_name` field is the operator-facing knob for partitioning. Two workflows that share a handler set but are deemed unrelated for scheduling purposes — say, fast LLM calls and slow PBF imports — can be assigned to separate lists. Runners are started with `--task-list <name>`; submission paths read a prefix-match configuration (`AFL_WORKFLOW_TASK_LIST_MAP=osm.=osm,anthropic.=anthropic`) and stamp the bootstrap task with the resolved list. Child event tasks created during execution inherit the orchestrating runner's list, so all tasks belonging to one workflow execution remain on the same partition by construction. Partitioning is therefore declarative at the configuration layer and requires no code change at the workflow or handler level.

The partitioning is preserved end-to-end by three properties: bootstrap tasks are stamped at submission time using the resolver against the qualified workflow name; child event tasks created by completion handlers read the same configuration in the runner's process; and the per-document atomicity of the claim ensures that exactly one runner — necessarily one whose `--task-list` matches the stamped value — ever dispatches a given task.

Task creation is also lock-free. The dual of the claim — task creation — has the same lock-free property. Each task carries a unique `uuid` generated by the producer, and is written through `save_task` as a single `replace_one(upsert=True)`. The orchestrating runner is the sole writer for the step tasks it generates from its own workflow execution; bootstrap and resume tasks are written once by their submitters. No two writers compete for the same document, so there is no producer-side contention to coordinate around either.

Implications for the coordinator-free thesis. A natural objection to coordinator-free designs is that they trade simplicity for the inability to partition or prioritise. Facetwork's partitioning shows that the trade is not forced: a coordinator was never needed for routing, because routing reduces to a query filter that the database evaluates atomically per claim. The same is true of recovery (§5.5), correctness (§5.6), and block-mediated advancement (§5.8). The pattern generalises: every place where a classical design would reach for a coordinator, Facetwork instead reaches for the database's per-document atomicity primitive, which it pays for once and reuses everywhere.

The cost is paid in two places. First, the database carries every claim request, every continuation, every state transition — its throughput must scale with the fleet, and an under-provisioned cluster bottlenecks the whole system. In practice, MongoDB's compound indices on `(state, name, task_list_name)` make per-list claims cheap enough that the bottleneck shows up only at fleet sizes well beyond the design target. Second, the rules of partitioning are encoded in operator-facing configuration rather than runtime API; misconfiguration (a runner polling a list that no submission writes to, or vice versa) produces silent idleness rather than an error. The dashboard surfaces per-list task counts and per-runner claim statistics so misconfiguration is observable, but it is not enforced at startup. This is a deliberate operational simplification: the same flexibility that allows the operator to partition by namespace also allows ad-hoc partitions (one-off cleanup runners, isolated test queues) without coordinator changes.

5.4.1 Horizontal scaling within a partition

The partitioning model in this chapter pairs naturally with horizontal scaling *within* a partition: N identical replicas, all configured with the same `--task-list`, all running the same handler set. The claim protocol does not change. Each replica polls independently; each claim is the same atomic `find_one_and_update`; the partial unique index on `(step_id, state="running")` ensures that even under contention the database admits exactly one claimer per task. Adding a replica is therefore equivalent to adding another concurrent caller of the claim primitive — no leader election, no in-fleet handshake, no schema change.

In practice the throughput scales linearly with replicas up to the point where the database becomes the bottleneck. A small empirical check is illustrative. With three `osm-geocoder` replicas polling a single `osm` list, ten concurrent `AnalyzeRegion` runs against the Liechtenstein extract dispatched 330 step tasks across the fleet, distributed 112 / 109 / 109 (34% / 33% / 33%). The expected distribution is uniform, because every replica's claim is symmetric: same query, same atomicity guarantee, same race conditions. The measured distribution is within 1% of uniform, with no fairness mechanism beyond per-document atomicity. The slight asymmetry comes from a small variance in poll-loop phase: a replica that wins a claim early in a cycle finishes its handler, returns to the poll loop, and sometimes wins the next claim before its peers have re-polled. The variance is bounded by the ratio of handler duration to poll period, and it converges to uniform as either ratio narrows.

There is no fairness guarantee — a single very-fast replica could in principle out-claim its peers — but in workloads where handler durations exceed the poll period (the common case), the imbalance is small and self-correcting. For workloads where strict fairness matters, the operator's recourse is to push concurrency further down into the handler (smaller granularity → more atomic claims → smaller imbalance) rather than to introduce a scheduler. The author has not yet encountered a workload where this trade was unsatisfactory in practice.

A small but real operational benefit follows from replicas being symmetric and unnamed: an operator can scale up or down in place without coordinating with the running fleet. `docker compose up -d --scale runner-osm-geocoder=3` brings two new replicas online; each registers itself with MongoDB, immediately starts polling, and begins claiming tasks. `--scale runner-osm-geocoder=1` removes two; their in-flight tasks fail their lease and are reclaimed under §5.5's protocol. There is no "drain" step, no leader transfer, no membership change to gossip. The fleet is whatever set of replicas is currently heartbeating to the servers collection.

5.5 Lease-based reclamation

If a runner claims a task and then dies — kernel panic, OOM, power failure, a human kicking the plug — the task is lost. Recovery is the dual of claiming: after a timeout, any runner may reclaim the task by issuing a similar `find_one_and_update` that matches `state = "running"` and `lease_expires < now`:

```
doc = tasks.find_one_and_update(
  {
    "state": "running",
    "name": {"$in": eligible_task_names},
    "task_list_name": self.task_list,
    "lease_expires": {"$lt": now, "$gt": 0},
  },
  {"$set": {
    "state": "running",
    "updated": now,
    "lease_expires": now + lease_ms,
    "server_id": self.server_id,
  }},
  return_document=ReturnDocument.AFTER,
)
```

Notice that this is a state-to-state transition: `running` → `running`. The atomic update ensures that even if two runners see the expired lease concurrently, only one successfully overwrites it (because the `find_one_and_update` is atomic over the single document and the `lease_expires` field value seen by the first runner would no longer match after the second runner's update). The partial unique index makes the dual guarantee: no two documents can both be `running` for the same `step_id`.

The handler renews its own lease by heartbeating. The heartbeat endpoint on the store writes both `task_heartbeat` and `lease_expires = now + lease_ms`. A handler that keeps heartbeating keeps its lease; a handler that stops — whether because it is stuck or because the process has died — lets its lease expire.

An empirical check confirms the protocol end-to-end. Running three `osm-geocoder` replicas, the author submitted a workflow and `docker kill`ed the replica that had just claimed the first task. The kill landed at $t = 2$ s after submission; the workflow ran to completion at $t = 129$ s, distributed across the two surviving replicas (17 and 16 tasks respectively, with zero tasks credited to the killed replica). The 127-second gap matches the configured reaper timeout (`AFL_REAPER_TIMEOUT_MS = 120 s`) plus a few seconds for the survivor to claim, dispatch, and complete. Recovery requires no operator intervention and no coordination among the survivors — the reaper sweep is a single atomic update against the orphan task, run periodically by every runner.

5.6 Correctness arguments

Safety. At most one handler executes a given task at a given time. The argument is as follows. A task is *executable* when its state is `running` and its lease is unexpired. The partial unique index ensures that at any instant, at most one document exists with state `running` per `(step_id)`. The atomicity of `find_one_and_update` ensures that no two runners can concurrently transition that document from pending to running. The atomicity of lease reclamation similarly ensures that no two runners can concurrently overwrite `server_id` on an expired lease. Therefore, at any instant, at most one runner *believes* it holds the task. In particular, the previous holder's `server_id` has been overwritten; its subsequent writes will either be rejected (if the update filter includes the old `server_id`, which Facetwork's heartbeat update does implicitly via the `state = "running"` match on the *current* document) or be harmless (if they write stale progress fields that will be overwritten).

A stress test against the running system confirms the safety property under contention. Ten `osm-geocoder` replicas all polling the same task list, 100 concurrent workflows submitted within a 2-second window, produced approximately 3 300 distinct step-level task documents. All 100 bootstrap (`fw:execute:<Workflow>`) tasks completed cleanly. The 10 replicas claimed between 257 and 339 step tasks each — a 1.32× spread, indistinguishable from the per-poll first-come race that drives the distribution. (An earlier iteration of this experiment showed a 25× spread; that was caused by a stable-UUID seed bug on container restart, since fixed in `seed_example_flows`. The bug's mode of failure — bootstrap tasks aborting with "Flow not found" — is the kind of orthogonal issue that empirical work tends to surface, and it tells us nothing about the claim race itself.) Across the entire run, **zero step-level `step_id` values reached a `completed` state through more than one task document**. The partial unique index on `(step_id, state="running")` held perfectly throughout, which is exactly what the safety argument claims. (The lease-reclaim case in the next paragraph remains a theoretical risk — the writeback path is not server-id-gated by construction — but it did not surface in this run.)

There is a subtler issue: the previous holder may still be executing when its lease expires. Facetwork cannot, and does not, kill the previous holder's process — it may be blocked in a C extension that ignores Python's `Future.cancel()`. Instead, Facetwork prevents the previous holder's *results* from affecting the system after reclamation. The handler's terminal write of task state goes through `save_task_if_owned`, which is a `replace_one` filtered on `{uuid, server_id == expected}` with `upsert=False`. If the reclaimer has overwritten `server_id`, the previous holder's write is silently dropped (`matched_count == 0`). The handler logs a warning and exits; the reclaimer's work — already in flight — produces the canonical result. For non-idempotent operations (sending a payment, posting a comment), the handler is still expected either to be idempotent at the external level (idempotency keys) or to encode a two-phase commit, because the *external action* is what we cannot un-perform; but the internal task state can no longer disagree with itself.

A second stress test exercises this directly. Ten replicas, 100 concurrent workflows, **three replicas killed at $t = 30$ s** (after all workflows were clearly in flight). The seven survivors absorbed the orphaned tasks via the lease reclaim protocol of this section; 99 of 100 workflows ran to completion. The one stuck workflow had its bootstrap claimed by a killed replica between starting `evaluator.execute()` and creating the first batch of child tasks — a separate edge case in workflow-level recovery, not in the lease protocol. Across the run, the **duplicate-completion count remained at zero**: the ownership-gated terminal write demonstrably prevents the lease-reclaim writeback race that the previous paragraph describes.

Liveness. Under the assumption of eventual synchrony (arbitrarily long but bounded message and processing delays eventually prevail), the system makes progress. If a task is pending and at least one live runner matches its name, that runner's next claim attempt will succeed. If a task is running but its holder has died, the lease will eventually expire and some live runner will reclaim. If every attempt fails indefinitely, either every runner is dead (no liveness is possible) or the database is unreachable (no liveness is possible). Under partial synchrony with at least one live runner and a reachable database, progress is guaranteed.

Fairness. Facetwork does not guarantee any particular fairness property across tasks. A greedy runner can monopolise a task list. In practice, per-runner concurrency limits (a thread pool of fixed size) prevent one runner from starving others, but I do not claim mathematical fairness.

5.7 Why no coordinator?

Classical distributed-system design instinct reaches for a coordinator: a leader process that maintains authoritative state, hands out work, and adjudicates conflicts. Coordinators simplify the reasoning: there is exactly one actor deciding each thing, and its decisions are canonical.

Coordinators also fail, and when they fail, they take the system with them. The practical cost of a coordinator is threefold:

1. **Single point of failure.** A coordinator crash stops new work. Mitigations (replicated coordinators, leader election) add their own operational complexity.
2. **Scaling bottleneck.** A coordinator that handles every claim must process claim-rate requests per second; at high rates, this dominates the throughput.
3. **Deployment rigidity.** Updating the coordinator requires either downtime or blue-green rollover, because the coordinator's local state must be preserved across restart.

Facetwork has no coordinator. The operational cost is zero: there is no service to update, no leader election to debug, no split-brain to worry about. The database does the coordination, and the database is already replicated and live-maintained by the DBA team.

This decision is not free. The main cost is that Facetwork relies on MongoDB's atomicity guarantees, specifically `find_one_and_update` and partial unique indices. A Facetwork port to a database without these primitives would be an undertaking. PostgreSQL with `SELECT ... FOR UPDATE SKIP LOCKED` would work; so would CockroachDB; so would FoundationDB. Systems without document-atomic compare-and-swap would require a different approach entirely — likely a coordinator after all.

5.8 Block-mediated step advancement

The claim protocol in §5.3 governs how individual tasks move from `pending` to `running`. A second coordination question arises above it: when a task completes, what decides which *next* tasks are created? The concurrency-by-default semantics of §4.4 sharpen the question. In a workflow that has just finished two parallel `Download` facets, some component must decide that both prerequisites of `Compare` are now satisfied and that `Compare` should be enqueued. Which component, and how?

A natural-seeming answer is: the runner that just finished the task. On completing step `ca`, the runner evaluates the containing `andThen` block, observes that `wa` is also complete, determines that `Compare` is now ready, and creates its task. This approach has an obvious flaw. If `ca` and `wa` complete simultaneously on different runners, both runners evaluate the same block at the same time; both observe that the block is now fully satisfied; both attempt to create the `Compare` task. The resulting race must be prevented, either by a lock across runners (reintroducing a coordinator) or by another atomic primitive (duplicating the claim-protocol machinery at a higher level, with all the implementation and correctness cost that entails).

Facetwork takes a different approach, and it is the key insight of this chapter. **Completed steps do not themselves evaluate the block. They emit a `continue` message to the containing block.** The block — more precisely, the block's evaluator in the workflow graph — processes continue messages **one at a time**. Because a block has at most one evaluator active on its behalf at any instant, the concurrency at the point where "what's next" is decided is *zero*. There is nothing to race.

5.7.1 Mechanism

The workflow evaluator is itself identified with a task (`fw:resume:<FacetName>` in the runtime, reserved under the `fw:` namespace; user code cannot forge these). Continue messages are enqueued as claimable tasks targeted at that evaluator. The ordinary claim protocol of §5.3 applies: exactly one runner claims each continue task. The claiming runner becomes the evaluator for one pass: it reads the current state of the block from the database, decides what (if anything) is newly ready, enqueues those tasks as pending, and marks itself complete.

The subtle case is when multiple steps complete in quick succession. Each emits its own continue message; several continue tasks may sit pending for the same block. When the first is claimed and processed, it may observe that *all* prerequisite steps are already complete and correctly enqueue the full next wave. A second runner then claims the next continue message, reads the now-updated block state, observes that nothing new is ready (because the previous evaluator already handled it), and terminates as a no-op. This is safe — no work is done twice — and it is cheap: a no-op evaluator is a single database read against the block state. In practice, step completions cluster within tens of milliseconds, and one productive evaluator pass coalesces many completions into a single dispatch wave, with any redundant continue messages handled as trivial no-ops thereafter.

5.7.2 The pattern, generalised

What this describes is precisely the **actor model** [Hew73, Agh86] applied to workflow evaluation. The block is an actor. Continue messages are mailbox messages. The actor processes its mailbox serially. Redundant messages become idempotent no-ops. The actor's mailbox discipline — exactly one message in flight at a time — is enforced not by an in-memory lock but by the claim protocol itself: only one runner ever holds the claim to the block's evaluator task. The lock-freedom of Facetwork's coordination extends cleanly from the individual task to the block, and by induction to the entire workflow graph.

The advantage over the each-runner-decides-what's-next alternative is that the race condition simply cannot occur: only one runner ever evaluates a given block at a time. The advantage over an explicit lock-based alternative is that the serialisation is implicit in the claim protocol — no extra mutex, no coordinator process, no leader election per block. The same atomic `find_one_and_update` that guarantees exactly-one execution of a leaf task also, by being applied to the evaluator task, guarantees exactly-one evaluation of the block at a time.

The pattern has an additional observability benefit. The continue message is a first-class task visible in the dashboard; an operator can see exactly when a block was last evaluated, what state it was in, and whether there are continue messages still pending. A lock-based alternative would hide this information inside the held-by metadata of the lock. Facetwork makes the evaluation pipeline itself observable, consistent with the thesis's broader commitment to state-centric visibility.

5.7.3 Safety and liveness

Safety. The invariant we maintain is: at any instant, at most one evaluator is actively computing "which tasks should be created next" for a given block. This follows from the claim protocol's per-task exactly-one-claim guarantee applied to the block's `fw:resume:<FacetName>` task. Redundant continue messages are safe because each evaluator's operation is read-modify-enqueue against the canonical block state in the database, and because newly-enqueued tasks are uniquely keyed on `step_id` — an evaluator that attempts to enqueue a step whose task already exists is a no-op at the store level, by construction.

Liveness. A continue message that is enqueued is eventually claimed under the liveness guarantees of the claim protocol (§5.6). Each evaluator makes progress: it either enqueues new tasks or terminates as a no-op. Continue messages therefore do not accumulate indefinitely. In the worst case, a block with k prerequisite steps completing simultaneously produces k continue messages, of which one is productive and $k-1$ are no-ops. The cost is linear in the fan-in of the block, which is bounded by the workflow author's design — and amortised over productive work that had to happen anyway.

Bounded staleness. There is one timing subtlety worth naming. An evaluator reads the block state at the moment it claims its continue task. If a prerequisite step completes *after* the read but before the evaluator finishes enqueueing, that completion will itself have emitted a continue message that is already queued behind the current evaluator. The current evaluator will not see the late completion, but its successor will, and will correctly handle it. The system never misses a completion; it only occasionally splits the handling of a wave of completions across two evaluator passes. This is the actor model's standard staleness property, and it is both sound and operationally benign.

5.7.4 A note on choice of mechanism

There are many possible implementations of "serialise work per block." Facetwork's choice — a per-block evaluator task claimed through the same protocol as any other task — has the virtue of reusing machinery that has to exist anyway. No new lock service, no new queue, no new primitive. The workflow graph itself is the coordination structure; the claim protocol is the synchronisation primitive; the database is the arbiter. Adding block-mediated advancement cost zero new infrastructure, and in the author's experience that economy is among the most satisfying design outcomes of the project.

5.9 Task resumption and the resume protocol

A sub-case of the claim protocol handles external agents that execute a facet *outside* the runner fleet. A Scala agent on a JVM host may be the canonical implementation of `osm.ops.SpatialIndex`. Facetwork's runner claims the task, serialises the parameters, and invokes an external process; when the external process completes, it writes the result back to the store and emits a resume task (`fw:resume:<FacetName>`) that the runner picks up. This elegantly turns the external process into a participant in the same claim protocol without requiring it to embed the Python runtime.

The resume task is special: its handler is not a user handler but a runtime method that unpacks the result, updates the step state, and continues workflow evaluation. The `fw:` prefix is reserved; user code cannot create `fw:*` tasks. This prefix reservation is enforced at schema registration time.

Chapter 6. Handlers, Registration, and Live Deployment

6.1 Handler registration

A handler is a Python function (or equivalent in another language) bound to a qualified facet name. Registration creates a document in the `handler_registrations` collection:

```
facet_name: str
module_uri: str         # file://path/to/module.py
entrypoint: str         # name of the callable
timeout_ms: int         # per-handler timeout (0 = global)
metadata: dict
```

The registration is read by every runner on startup. When a runner is asked to handle `osm.ops.PostGisImport`, it looks up the registration, dynamically imports the module, resolves the entrypoint, and invokes it with the task's parameters plus a set of injected callbacks (`_step_log`, `_task_heartbeat`, `_set_stage_budget`, and so on).

The **crucial design decision** is that registration lives in the database, not in the runner's code. A new handler is deployed by:

1. Pushing the module code to the runners' file systems (via `rsync`, Ansible, or Kubernetes configmap).
2. Calling `register_handler(...)` through the MCP server or the dashboard or a CLI script.

Step 2 takes effect on every runner that re-reads registrations (done on each poll cycle, a few times per second). There is no fleet-wide restart.

6.2 Handler invocation contract

The injected payload a handler receives contains:

- The typed parameters declared by the facet.
- `_step_log`: a callback for emitting log entries visible in the dashboard.
- `_task_heartbeat`: a callback for signalling liveness and optionally reporting progress percentage.
- `_set_stage_budget`: a callback for declaring a stage's timeout budget (Chapter 8).
- `_task_uuid`, `_retry_count`, `_is_retry`: metadata so the handler can detect retry scenarios.
- `_ctx`: a `HandlerContext` object that wraps the above in a typed interface.

A handler returns a dictionary conforming to the facet's declared return schema. The runtime validates the return against the schema; a mismatch fails the step with a specific error.

6.3 The `HandlerContext`

A handler that takes the typed context route looks like this:

```
def handle(payload: dict) -> dict:
    ctx = HandlerContext.from_payload(payload)

    if ctx.is_retry:
        ctx.step_log(f"Retry #{ctx.retry_count}: checking prior work")

    with ctx.stage("pbf_scan", timeout_ms=scan_budget_ms) as s:
        collector = CombinedCollector(..., heartbeat=s.heartbeat)
        collector.apply_file(pbf_path, locations=True)

    return {"result": ...}
```

The context provides four capabilities in a single typed object: liveness signalling, structured logging, stage budget declaration, and retry introspection. Earlier versions of Facetwork exposed these as flat keys on the payload dict; the context object was introduced to improve discoverability and to provide a stable interface that could evolve without breaking every handler.

6.4 Handlers in the supported languages

Because the handler contract is described at the protocol level — a named facet, a parameter dict, a returned dict of named output fields — it is portable across language runtimes. Facetwork ships client libraries for **Python**, **Go**, **Java**, **Scala**, and **TypeScript**, all of which implement the same poll / claim / execute / yield / heartbeat cycle against the same MongoDB-backed task collection. An author writes a handler in whichever language fits the problem; the FFL workflow that references the handler by name is identical regardless of the language it eventually binds to.

The canonical "hello-world" handler in each supported language is small enough to fit on one page. All five versions implement the same facet: `ns.MyFacet`, taking one `input: String` parameter and returning `{ result: <input> + " processed" }`.

Python (RegistryRunner, in-process):

```python
from facetwork.runtime import RegistryRunner

runner = RegistryRunner.from_environment()

@runner.handler("ns.MyFacet")
def my_facet(payload: dict) -> dict:
    return {"result": payload["input"] + " processed"}

runner.run()
```

Go:

```go
package main

import (
    "context"
    "log"

    aflagent "github.com/facetwork/fw-agent"
)

func main() {
    cfg := aflagent.FromEnvironment()
    poller := aflagent.NewAgentPoller(cfg)

    poller.Register("ns.MyFacet", func(params map[string]interface{})
        (map[string]interface{}, error) {
        input := params["input"].(string)
        return map[string]interface{}{"result": input + " processed"}, nil
    })

    if err := poller.Start(context.Background()); err != nil {
        log.Fatal(err)
    }
}
```

Java:

```java
import afl.agent.AgentPoller;
import afl.agent.AgentPollerConfig;
```

```java
import java.util.Map;

public class MyAgent {
    public static void main(String[] args) throws Exception {
        AgentPoller poller = new AgentPoller(AgentPollerConfig.fromEnvironment());

        poller.register("ns.MyFacet", params -> {
            String input = (String) params.get("input");
            return Map.of("result", input + " processed");
        });

        poller.start();
    }
}
```

Scala:

```scala
import afl.agent.{AgentPoller, AgentPollerConfig}

@main def run(): Unit =
  val poller = AgentPoller(AgentPollerConfig.fromEnvironment())

  poller.register("ns.MyFacet") { params =>
    val input = params("input").toString
    Map("result" -> (input + " processed"))
  }

  poller.start()
```

TypeScript:

```typescript
import { AgentPoller, resolveConfig, Handler } from "@afl/agent";

const poller = new AgentPoller(resolveConfig());

const myHandler: Handler = async (params) => ({
  result: (params.input as string) + " processed",
});

poller.register("ns.MyFacet", myHandler);
await poller.start();
```

Four features are worth noticing across these examples.

First, the handler body is always about the problem, not the transport. In every language, the handler is a function from a parameter map to a result map. The framework deals with the MongoDB claim protocol, the lease renewal, the heartbeat, the retry machinery, and the workflow-evaluator resume message — none of that leaks into the author's code. An engineer writing their first Facetwork handler has no protocol knowledge to acquire beyond "take a dict, return a dict".

Second, the registration is by qualified facet name. None of the examples mentions a workflow. The handler does not know which workflows call it, which parameters the caller might have filled from which upstream step, or which sibling `andThen` block it will sit in. That separation is what §4.10 argued for at the language level; here it is what the handler contract delivers at the runtime level. A handler can be written, registered, and exercised in isolation; FFL authors compose it later without touching its code.

Third, the shape is identical across languages. Go, Java, Scala, and TypeScript use the **AgentPoller** model — a standalone process connected to the same MongoDB instance as the Python runners, claiming tasks whose registered facet name matches those it has registered. Python has the additional **RegistryRunner** model, which shares the process with other Python handlers and avoids an extra hop, but the handler function itself looks the same. The choice between AgentPoller and RegistryRunner is a deployment decision, invisible to handler and workflow authors.

Fourth, the configuration is uniform. All five clients resolve their MongoDB connection through the same chain — an explicit config path, the `AFL_CONFIG` environment variable, an `afl.config.json` file in standard locations, direct `AFL_MONGODB_URL` / `AFL_MONGODB_DATABASE` environment variables, and finally built-in defaults. A team running handlers in multiple languages configures them all the same way, which matters more in production than any single one of the language-level details.

A handler in a more sophisticated handler — one that uses the typed context — reaches for the language's equivalent of `ctx.step_log`, `ctx.heartbeat`, and `ctx.stage(...)`. The Python API is the canonical form, with matching idioms in the other four clients. A long-running Scala handler that wants to declare a PBF-scan stage budget writes:

```
poller.register("osm.ops.PostGisImport") { (params, ctx) =>
  val fileSize = params("file_size_bytes").asInstanceOf[Long]
  ctx.stage("pbf_scan", timeoutMs = math.max(30 * 60_000L, fileSize * 20L / 1_000_000))
        { s =>
    val result = scanPbf(params, onProgress = s.heartbeat)
    Map("result" -> result)
  }
}
```

with exactly the same behaviour the Python `with ctx.stage(...)` context manager delivers. The portability of the handler contract, down to the richer parts of the context, is one of the quieter but most consequential design properties of Facetwork: a team can change its mind about what language a given facet is implemented in, without the workflow that uses it ever being rewritten.

6.5 Agent models

Facetwork supports four handler execution models, each with a distinct operational shape:

1. **RegistryRunner**: the default, Python-only. The runner process reads registrations from the database and loads handler modules dynamically. This is the highest-throughput, lowest-latency model, suitable for handlers that fit comfortably in the Python runtime.
2. **AgentPoller**: the external-process model. A standalone agent service polls for tasks matching registered facets and returns results. This model is used when the handler cannot be run in the Python runtime, for example because it is written in Scala and runs on the JVM, or because it requires specific native libraries or CUDA support. AgentPoller provides multi-language client libraries that handle the poll/execute/resume cycle.
3. **RunnerService**: a heavier internal runner with per-task thread pools, heartbeat daemons, and explicit lifecycle management. Used for long-running, memory-intensive tasks that need isolation.
4. **ClaudeAgentRunner**: handlers whose implementation is an LLM call. The runner dispatches to the Anthropic API based on a declared prompt block in FFL. This model is both a convenience (no Python code needed for LLM-backed facets) and a policy enforcement point (every LLM call goes through a single rate-limit, cost-accounting, and audit pipeline).

The decomposition into four models is an operational decision; from the FFL author's perspective, it is invisible. A facet is a facet. The choice of agent model is made at handler registration time.

6.6 Graceful drain and quarantine

Two operational primitives distinguish Facetwork's fleet management from that of most workflow systems.

Drain. `scripts/drain-runners` stops runners and resets their in-flight tasks back to pending. The tasks are picked up by other runners; no work is lost. Drain emits a step log for each reset task, so that the audit trail records *why* a task was re-executed. Drain is the standard operational tool for fleet upgrades and planned maintenance.

Quarantine. A quarantined runner stays alive — it keeps heartbeating, it renews task leases for work it has already claimed — but it stops claiming *new* tasks. This is the right tool when an operator suspects a specific server is misbehaving and wants to stop it acquiring more work without killing its in-flight tasks. The runner can be un-quarantined to resume normal operation, or the operator can drain it after its current tasks finish. Quarantine is implemented as a per-server state (`ServerState.QUARANTINE`) stored in the `servers` collection; the runner's claim loop checks this state on each poll cycle.

The combination of drain and quarantine gives operators three lifecycle levers — running, quarantine, shutdown — rather than the binary alive/dead of most systems. The distinction matters because the recovery cost of a reclaimed task is not zero: a task that has run for thirty minutes and is interrupted must restart from scratch in the next claim (unless handlers encode their own checkpointing, which few do). Quarantine lets operators wait for natural task completion rather than forcing interruption.

6.7 Rolling deployment

A rolling deployment in Facetwork follows this recipe:

1. Push the new handler code to all runners.
2. For each runner (or a subset at a time): a. Quarantine the runner. b. Wait for its in-flight tasks to complete. c. Stop the runner. d. Restart the runner with the new code loaded. e. Un-quarantine the runner.

Alternatively, for handlers whose implementation has not changed but whose registration metadata has (for example, a per-handler timeout adjustment), step 2 is not needed at all: the `register_handler` call is enough.

This recipe is implemented in `scripts/rolling-deploy`. The interesting property is that the fleet's total throughput decreases during the deploy but never reaches zero: at any instant, some subset of runners is available to claim work. A fleet of ten runners doing a rolling deploy with one-at-a-time replacement maintains nine tenths of its throughput throughout; compare this with a Jenkins controller restart, which is a hard zero.

6.8 Fleet inspection

A distinguishing feature of Facetwork's operational model is that the fleet is genuinely observable. `scripts/list-runners` produces a live view of every registered runner, its state, its ping time, its currently-claimed tasks, and its loaded handlers. The dashboard's `/v2/servers` page renders the same information graphically, with quarantine toggles on each row. The combination of a live, inspectable fleet and per-server lifecycle primitives turns fleet operations from a black art into a visible, scriptable practice.

6.9 Examples as standalone pip packages

The handler-registration model of §6.3 is agnostic about *where* the registered code lives. The framework finds handlers via a Python entry-point group — `facetwork.examples` — and any package that declares one is discovered, loaded, and registered without changes to the framework. Internal `examples/<name>/` directories use this same mechanism; nothing about them is privileged.

In practice this has let entire example packages graduate out of the framework monorepo into their own pip-installable repositories. At the time of writing there are eight such packages: `fwh_osm` (OSM extraction, the canonical case study of Chapter 13), `fwh_osm_lz` (a pure FFL workflow catalog that consumes `fwh_osm`'s facets without contributing any handlers of its own), `fwh_noaa_weather`, `fwh_jenkins`, `fwh_census_us`, `fwh_genomics`, `fwh_sensor_monitoring`, and `fwh_anthropic` (a multi-area wrapper around the public surfaces at github.com/anthropics — Messages, Batch, Files, Agent SDK, Claude Code, Computer Use). Each is installable with one `pip install -e` and discoverable by every Facetwork runner via the entry point.

The shape that emerges is that examples are not example *folders* but example *packages*. A team that wants to ship a domain to production builds a `fwh_<domain>` repo with the same internal layout as the in-repo examples — `ffl/`, `handlers/`, `tools/_lib/`, `tests/` — adds a `pyproject.toml` declaring the entry point, and is operationally indistinguishable from the in-tree case from the framework's perspective. The development workflow this enables is also worth noting: a single Docker Compose file in the framework repository (`docker-compose.full-stack.yml`) starts one dedicated runner per `fwh_*` package, each bind-mounting its source directory and pip-installing it editable at container start, so that handler edits land in the running runner without a rebuild. The framework provides the entry-point discovery; the example packages provide their own dependency closure (the OSM package pulls `osmium` and `pyproj`, the Anthropic package pulls `anthropic` and optional SDK extras), and the framework remains free of any of them.

This is, in a small way, the original §4.10 argument paying off downstream: because workflows reference facets by qualified name and handlers register themselves into a typed registry, the boundary between framework-code and example-code is something operators can move at will without disturbing FFL authors. The same workflow that referenced an in-tree `osm.ops.PostGisImport` last year today refers to a facet shipped from a separate Git repository on a separate release cadence; the FFL source did not change.

Chapter 7. Recovery Without Replay

7.1 Two recovery traditions

There are two ways to recover a distributed workflow after failure, and the choice between them drives much of the rest of the system design.

Replay. The canonical recovery mechanism in Temporal and Cadence. Every decision made by the workflow is recorded in an event history. When a worker resumes a workflow (because the previous worker died), it fetches the full history and re-executes the workflow code, feeding in events one at a time, until it reaches the point where the history ends; it then continues from there. The price is determinism: the workflow code must produce the same decisions when fed the same history, which means the code cannot use the current wall clock, cannot invoke random number generators directly, cannot iterate over maps with non-deterministic order, and so on. Systems that use replay typically provide sandboxed alternatives for these operations (`workflow.now()`, `workflow.random()`), but the constraint remains and is deeply felt.

State. The canonical recovery mechanism in Camunda, Airflow, and Facetwork. The current state of each step (pending, running, completed, errored) is written to the database on every transition. When a worker resumes, it reads the state and decides what to do next. There is no replay, no history reconstruction, no determinism constraint. The price is that the workflow description must be explicit about what state needs to survive — essentially, every piece of data that informs a future decision must flow through a facet's typed return value.

Facetwork is firmly in the state-recovery tradition. The design argument for this position is threefold.

7.2 Why state wins for long-running work

First, the determinism constraint is incompatible with a large and important class of real workloads. The OSM import workflow, our motivating example, reads the size of a downloaded file (non-deterministic: the file may be different between runs), queries the current row count in the database (non-deterministic: other concurrent writers may have changed it), and times an external `COPY` operation (non-deterministic under any reasonable model). Rewriting this handler to thread everything through a deterministic activity-call model would be possible but would substantially complicate the code and the mental model. More importantly, the *whole point* of a state-persisted model is that the handler does not need to know it might be replayed — because it will not be.

Second, the recovery granularity is step-level, not workflow-level. In a replay system, recovery necessarily reconstructs the whole history. This is linear in the number of events, which for long workflows is thousands of events and significant CPU. In a state system, recovery is a single database lookup: *what is the state of this step?* Followed by *are there downstream steps that need to run?* The dashboard's `Retry`, `Retry All Errors`, `Reset Block`, and `Re-run From Here` actions operate at the step level — the operator can re-run exactly the step that failed, re-run a whole sub-block, or re-run from a point and discard everything downstream. In a replay system, the operator's options are more limited: either restart from the top or patch the history manually, both unpleasant.

Third, observability is easier. A state-persisted workflow's current status is a single row in a database. A replay-system workflow's current status requires interpreting an event history, a job that dashboards handle by essentially running the replay themselves. Debugging a step that failed in a state-persisted system means looking at that step's row. Debugging in a replay system means reading the history up to the failure point, which is harder.

7.3 The cost of state recovery

State recovery has real costs that I will not hide.

Idempotence is the handler author's problem. If a handler sends a payment to an external system and then crashes, a state-recovery system will re-run the handler, potentially sending the payment twice. The handler author must either make the handler idempotent (typically by threading an idempotency key through to the external API) or implement manual cleanup on retry. Replay systems dodge this by ensuring that external calls happen in "activities" that are recorded exactly once.

State evolution is harder. If the state schema for a step changes between Facetwork versions, migrations must be applied to the database. Replay systems handle schema changes somewhat more gracefully because the workflow code is what changes; the history format is more stable.

Undocumented dependencies on mutable state can lurk. A handler that reads a configuration value on first execution and acts on it — then gets reclaimed and reads a different value on second execution — will behave inconsistently. Replay systems would replay the original read. State systems expect handlers to be read-your-own-writes clean.

These costs are real but manageable. For long-running, heterogeneous workflows they are, on balance, a favourable trade against the cost of the determinism constraint. For short, high-throughput workflows with strong exactly-once external semantics, the trade goes the other way, and Temporal or Cadence is the correct choice.

7.4 The workflow-repair mechanism

Facetwork's step-level recovery actions (`Retry`, `Re-run From Here`, `Reset Block`) are manual tools for operators, but there is also an automated repair mechanism. `scripts/repair-workflow` (or the equivalent dashboard button) diagnoses and fixes a stuck workflow through five checks:

1. **Runner state.** A workflow is marked completed but has non-terminal work; reset to running.
2. **Orphaned tasks.** Running tasks on dead or shutdown servers; reset to pending.
3. **Transient step errors.** Errors whose message matches network or connection patterns; retry (EventTransmit).
4. **Ancestor blocks.** Errored ancestors that need to be reset so downstream execution resumes.
5. **Inconsistent steps.** Steps marked complete but with failed tasks; reset to EventTransmit.

Repair is idempotent: running it twice on an already-repaired workflow is a no-op. Repair is also preventative: runners check their own completion status before marking a workflow as done, reducing the chance that state inconsistencies arise in the first place.

The existence of the repair tool is, in a sense, an admission that the state model can reach inconsistent states. I do not think this is a mark against the model; rather, it reflects the honest acknowledgement that distributed state transitions are subtle and that operators occasionally need a tool to restore invariants. The replay model hides this by reconstructing from history; when the history is itself inconsistent (a corrupted event, a ghost activity result), replay systems have their own tools for manual history patching, which are strictly less ergonomic than Facetwork's repair.

7.5 Step recovery semantics

The four dashboard recovery actions deserve explicit semantic treatment:

- **Retry.** Applies only to errored steps. Resets the step from `Errored` to `EventTransmit` (the state at which a handler is dispatched). Upstream data is unchanged. The handler re-runs with the original inputs.
- **Retry All Errors.** Applies to a block. Recursively finds every errored leaf step under the block and retries each.
- **Reset Block.** Applies to a block. Deletes all descendant steps, tasks, and logs under the block and restarts the block from scratch. Semantically: discard everything the block has done, do it again.
- **Re-run From Here.** Applies to any completed or errored step. Resets the step, clears its results, deletes all steps *downstream* of it (that is, steps that depend on its output), and re-executes from the step onwards. This is the operator's tool for patching a handler and re-running only the affected portion of a workflow; upstream work is preserved.

Together, these four actions cover the common operator needs: "something transient failed, try again", "this block is broken, start it over", "I changed the handler, re-run from here". Each action produces a step log entry recording the operator, time, and action type, so that the audit trail captures manual interventions.

Chapter 8. Staged Timeouts and Dynamic Budgets

8.1 The flat timeout problem

Most workflow systems treat timeouts as a single scalar per task. Airflow has `execution_timeout`; Celery has `soft_time_limit` and `time_limit`; Temporal has `StartToCloseTimeout`. Each is a single duration: if the task exceeds it, the task is killed.

This flat model is wrong for multi-stage handlers. Consider again the OSM import:

Stage	Typical duration (France-scale)
Prior-import check	< 1 s
Local PG setup	~10 s
PBF scan	2–6 h
Staging merge (local)	10–30 min
Transfer to main	30 min – 2 h
Main-table merge	10–45 min
Audit log	< 1 s

A flat timeout either must be at least 8 hours (the maximum plausible total), which means that a truly stuck handler runs for 8 hours before the watchdog intervenes, or it must be shorter than the PBF scan, in which case the handler dies partway. Neither is correct.

The correct treatment is: **each stage has its own timeout budget, chosen from the nature of that stage**.

8.2 The `stage()` context manager

Facetwork exposes staged timeouts through the handler context:

```
with ctx.stage("pbf_scan", timeout_ms=scan_budget_ms) as s:
    collector.apply_file(pbf_path, locations=True)

with ctx.stage("staging_merge", timeout_ms=merge_budget_ms) as s:
    merge_staging_to_main(conn, region)

with ctx.stage("transfer_to_main", timeout_ms=transfer_budget_ms) as s:
    copy_binary_stream(local_conn, main_conn, region)
```

On stage entry, the handler computes a budget (optionally from measured input size) and calls `ctx.stage()`. This invokes the injected `_set_stage_budget` callback, which writes `stage_budget_expires = now + budget_ms` on the task document and renews the lease. The runner's global watchdog, on its next poll, reads this field and treats the stage budget as an override: the task is killed only when *both* the global execution timeout and the stage deadline have elapsed.

On stage exit (normal or exceptional), the context manager clears the stage budget. The task is then subject to the normal global timeout.

8.3 Dynamic extension

A stage handle exposes `extend(extra_ms)`, which pushes the deadline out. This is used when the handler discovers, partway through, that the input is larger than estimated:

```
with ctx.stage("pbf_scan", timeout_ms=estimated_budget) as s:
    collector = CombinedCollector(..., heartbeat=s.heartbeat)
    for chunk in chunks:
        collector.process(chunk)
        if collector.scanned_more_than_expected():
            s.extend(30 * 60_000)  # add 30 minutes
```

The extension renews the budget and the lease; the watchdog re-reads on its next cycle. The total budget at any moment is the sum of the initial budget and all extensions to date; the stage handle tracks this for logging purposes.

8.4 Composition with heartbeats

Stage budgets do not replace heartbeats; they complement them. A heartbeat is a *liveness signal*: "I am still running." The runner's watchdog uses heartbeats to distinguish active handlers from stuck ones, but it cannot distinguish active-but-slow from active-and-making-progress. A stage budget declares: "This stage should take up to this long; do not kill me during it."

Best practice is:

- For fast stages (seconds): heartbeat only, rely on the global timeout.
- For medium stages (minutes to an hour): heartbeat frequently and declare a stage budget matching expected duration.
- For long stages (hours): heartbeat at regular intervals, declare a stage budget sized from input, and extend mid-stage if the input turns out larger than estimated.

The OSM PBF scan is the canonical long-stage case: budget is `max(30 min, file_size_mb × 20 s/MB)`, heartbeats fire per processed node batch, and the handler extends the budget if the pyosmium scan reports more nodes than the file size alone suggests.

8.5 Why the watchdog respects stage budgets

A natural question is: why not simply set a very long global timeout and rely on heartbeats? The answer is that a stuck handler may still be heartbeating — heartbeat is a background thread in some frameworks, and it can keep running after the main logic has deadlocked. The global timeout is the backstop: if a task has been running for more than *N* hours without observable progress, something is probably wrong. Stage budgets *extend* the global timeout for specific, pre-declared reasons; they do not replace it. A handler that declares a 48-hour stage budget and then deadlocks immediately will eventually trip either the global timeout (if heartbeats stop) or the stage deadline (if they continue to the end of the budget). Either way, the task is released.

This is the distinguishing property of Facetwork's staged-timeout model: it is **additive**, not *replacing*. The global timeout and the stage budget both apply; a task is killed only when neither can justify continued execution.

8.6 Environment-scoped defaults

The OSM importer declares stage budget defaults via environment variables, so that operational tuning does not require handler code changes:

```
AFL_OSM_SCAN_MS_PER_MB   = 20000         # 20 s per MB of PBF
AFL_OSM_SCAN_FLOOR_MS    = 1800000       # 30 min minimum
AFL_OSM_MERGE_MS_PER_MB  = 4000          # 4 s per MB
AFL_OSM_MERGE_FLOOR_MS   = 900000        # 15 min minimum
AFL_OSM_TRANSFER_MS_PER_MB = 2000
AFL_OSM_TRANSFER_FLOOR_MS  = 1800000
```

These defaults can be adjusted per-example through `runner.env` files, per-environment through the deployment configuration, or even per-task through FFL parameters. The flexibility is there; the default values encode what has worked empirically.

8.7 Compared with flat timeouts

Property	Flat timeout	Facetwork staged timeout
Handles fast+slow stages in one task	Awkward	Native
Adapts to input size	No	Yes (via heuristics or handler logic)
Mid-execution extension	No	Yes (`s.extend(extra_ms)`)
Preserves global safety net	—	Yes (global timeout still applies as ceiling)
Handler-visible	—	Yes (typed `stage()` context manager)
Dashboard-visible	—	Yes (stage name + budget rendered per task)

No system known to the author offers an equivalent construct in its public API. Temporal's `StartToCloseTimeout` is per-activity; one could approximate staged timeouts by breaking the handler into many activities, but this interacts badly with the determinism constraint (each activity call is a decision point whose order must be reproducible) and imposes substantial boilerplate. Airflow tasks have `execution_timeout` but the task boundary is an Airflow scheduling concept; splitting one long handler into five tasks changes the DAG shape. Camunda has timer boundary events but they are designed for human-in-the-loop waits, not for internal stage budgets within a service task.

Staged timeouts, then, are a small but genuine contribution: a composition of existing mechanisms (per-task timeouts, lease renewal, heartbeats) into a construct that handlers use naturally and that the runtime enforces correctly.

Part III — Comparison and Evaluation

Chapter 9. Temporal and the Determinism Tax

9.1 Temporal in brief

Temporal is the leading representative of the workflow-as-code, event-sourced recovery tradition. Workflows are written in a general-purpose language (Go, Java, TypeScript, Python, .NET); the Temporal server records every decision the workflow makes in an immutable event history; on failure, any replica can reconstruct the workflow's state by replaying the history deterministically.

The Temporal model's elegance is undeniable. The developer writes code that looks like an ordinary imperative program — `result = activity.do_thing(); if result: activity.do_other_thing()` — and the system handles all the distributed concerns transparently. A crashed worker resumes on another node; the workflow code never sees the crash.

9.2 The determinism constraint

Temporal workflows must be deterministic. Every decision the workflow code makes must be reproducible given the same event history. This means:

- No wall-clock time reads (`time.time()` is forbidden; use `workflow.now()`).
- No random numbers (`random.random()` is forbidden; use `workflow.random()`).
- No network calls directly (must be wrapped in activities).
- No iteration order over unordered collections (maps, sets).
- No unsynchronised global state.
- No blocking on external synchronisation primitives.

Temporal provides wrappers for the legitimate uses and a strict mode that detects most violations at development time. The constraint is enforced by the replay model itself: a workflow that violates it will simply produce wrong results on replay, which the server will detect by comparing the replayed decisions against the recorded history.

The constraint is non-trivial to satisfy. A handler written for a state-recovery system, translated naively to Temporal, will almost certainly violate determinism somewhere. Translation requires rethinking every source of non-determinism — which is many sources, in modern Python or Java code — and either replacing it with a deterministic equivalent or extracting it into an activity. The total engineering cost is substantial, and the resulting code is sometimes harder to read than the original: the mental overhead of "is this call a workflow call or an activity call" is non-zero.

9.3 When determinism pays off

Determinism is not a pure cost; it buys real things.

First, it buys **automatic recovery without operator intervention**. When a Temporal worker dies mid-workflow, another worker resumes the workflow exactly where the previous one left off. No step is re-executed, no side effect is duplicated, no history is lost. This is genuinely valuable for workloads where every side effect matters — payments, message sends, external-API calls with billable effects.

Second, it buys **time-travel debugging**. A developer can replay any workflow history locally, step through the code, and see exactly what decisions were made and why. This is an under-appreciated Temporal feature.

Third, it buys **schema-flexible persistence**. The recovery protocol does not care what the workflow's state looks like internally, only that the history determines it. Schema changes to internal workflow state do not require database migrations.

These benefits are real and have won Temporal a deserved place in many architectures.

9.4 When determinism loses

Determinism loses when the workflow's work is dominated by non-deterministic external interactions: operations whose results depend on time, on the current database state, on file sizes, on the contents of a message from an external service.

The OSM import is not a pathological case; it is typical for data processing, bioinformatics, media processing, and many scientific workflows. Each stage reads data from the outside world (the PBF file, the database, the filesystem) and makes decisions based on what it finds. Rewriting each decision as an activity call is possible but noisy, and the resulting code is dominated by activity boilerplate rather than by the logic the scientist or engineer is trying to express.

Facetwork's state-recovery model, by contrast, lets the handler read the world freely, makes the results available to downstream facets through typed return values, and delegates idempotence (where it matters) to the handler's external interaction. For long-running, data-heavy, non-deterministic workloads, this is a materially better fit.

9.5 The operational shape

Temporal requires a separately-deployed server: the Temporal frontend, history, matching, and worker services, backed by a datastore (PostgreSQL, MySQL, Cassandra). Running Temporal in production means running and operating a distributed system with its own operational characteristics, separate from the database the application uses for its own state.

Facetwork requires MongoDB. That is the operational shape. A team already running MongoDB for application purposes adds no new operational component; a team not already running MongoDB adds one, but the MongoDB ecosystem is mature and the operational knowledge is widely distributed.

For teams of any size, the Temporal operational cost is a non-trivial overhead. This is not a criticism of Temporal — a mature workflow system needs an infrastructure of its own — but it is a relevant consideration in the choice.

9.6 Table of contrasts

Dimension	Temporal	Facetwork
Workflow description	Code (Go, Java, TypeScript, Python, .NET)	FFL DSL
Code determinism constraint	Yes	No
Recovery mechanism	Event-sourced replay	State persistence + retry
External infrastructure	Temporal cluster + datastore	MongoDB only
Runtime handler updatability	Requires worker restart (usually)	Live via DB registration
Multi-language handlers	Yes, via Temporal SDKs	Yes, via AgentPoller
Timeout model	Flat per-activity	Staged with dynamic budgets
Compile-time topology check	Partial (via SDK)	Full (via FFL parser)
Primary workload fit	High-throughput, short, exactly-once	Long-running, heterogeneous, at-least-once

Temporal and Facetwork are not direct competitors in every workload. They are near-optimal answers to adjacent questions.

Chapter 10. Camunda, BPMN, and the Weight of Standards

10.1 Camunda in brief

Camunda is a process engine for BPMN 2.0. Workflows are drawn as BPMN diagrams in a modeller tool, serialised as XML, and executed by a process engine. The engine persists workflow state in a relational database (PostgreSQL, MySQL, Oracle) at every step boundary. Service tasks can invoke external workers through a job worker protocol.

Camunda has legitimate strengths. BPMN is an interchange standard: a diagram drawn in Camunda's modeller can be opened (in principle) in any BPMN 2.0 tool. The persisted state model is familiar to relational-database operators. The engine has been deployed at enterprise scale for years and has well-understood operational characteristics.

10.2 The standards cost

BPMN 2.0 is 538 pages of specification [OMG11]. The executable subset is smaller but still substantial. Conforming implementations must handle a long tail of esoteric elements — transaction subprocesses, compensation boundaries, event subprocesses, message correlation, signal throw events — that most users never touch but that every implementation must implement.

The practical effect is that Camunda, and BPMN in general, is a heavy system. Simple workflows are not simple to author; the minimum viable BPMN diagram includes a start event, some service tasks, gateways, and an end event, all drawn and wired. The FFL equivalent is three lines of code. For workflows that fit BPMN's sweet spot — enterprise business processes with human-in-the-loop tasks, timer-based escalations, and compensating transactions — the weight is justified. For workflows that are simply "do X, then Y, then Z", it is not.

10.3 XML as canonical form

BPMN diagrams are canonically XML. This means:

- Workflows are not readable in the terminal.
- Workflow diffs in pull requests are almost useless.
- Textual editors cannot productively edit workflows.
- Refactoring tools are limited to what the graphical modeller supports.

FFL's canonical form is source code. A pull request touching an FFL workflow is a normal code review. A diff shows exactly what changed. A grep across the workflows finds all uses of a given facet. These are not minor conveniences; they are the difference between workflow code that can be maintained by a software engineering team and workflow code that requires a dedicated BPMN modeller operator.

10.4 The graphical-first assumption

BPMN's design assumes that workflows are primarily designed and maintained in a graphical tool. For some audiences — especially non-programmers in large enterprises — this assumption holds. For software engineering teams, it does not. Engineers are better served by code; they read, review, refactor, and test code with much better tools than any graphical modeller provides.

Facetwork's dashboard renders workflow topology graphically — the operator can see the step dependency graph, the live state of each step, and the data flowing between them — but the canonical form is FFL source. The graphical view is a projection of the source, not the other way around. This is the right arrangement for software engineering teams.

10.5 State model similarities and differences

Under the hood, Camunda and Facetwork are more similar than different: both persist workflow state in a database, both support step-level retry, both have live observability dashboards. The differences are:

- **Language.** BPMN XML vs. FFL.
- **Coordination.** Camunda uses a job executor that polls the database; Facetwork uses atomic claim. The mechanisms are similar in principle; Facetwork's is more strictly lock-free and its claim semantics are documented at the primitive level. Camunda's are correct in practice but less crisply specified in the documentation.
- **Live updatability.** Camunda supports process definition versioning but changing a running definition is complex. Facetwork's handler registration model lets the *implementation* of a facet change without touching the workflow or restarting any runner.
- **Target workloads.** Camunda excels at human-in-the-loop enterprise processes. Facetwork excels at long-running data and compute workflows.

The right choice between Camunda and Facetwork depends on workload, not on technical merit: a bank reconciliation workflow with approval steps and audit requirements may well be correct in Camunda; a genomic pipeline with hour-long compute steps is almost certainly correct in Facetwork.

Chapter 11. Airflow and the Centralised Scheduler

11.1 Airflow in brief

Apache Airflow is the most widely deployed workflow system in the open-source data ecosystem. Workflows (DAGs) are written in Python; a centralised scheduler reads the DAGs, computes execution plans, and dispatches tasks to workers via executors (`SequentialExecutor`, `LocalExecutor`, `CeleryExecutor`, `KubernetesExecutor`, and others).

Airflow's cultural influence on the data-pipeline space is enormous. Its DAG-in-Python model has been copied by Prefect, Dagster, and many bespoke systems. Its operational model — scheduler plus executors plus database plus web UI — is the default mental model for modern data-engineering teams.

11.2 The centralised scheduler

The Airflow scheduler is a single process per deployment. It reads DAG definitions, evaluates schedule triggers, computes which tasks are runnable, and dispatches them. For years this was a hard single point of failure and a throughput bottleneck; the scheduler could only run one DAG evaluation at a time, and DAG complexity directly affected throughput.

Airflow 2.0 introduced scheduler high-availability: multiple schedulers can run concurrently, coordinating through the database. This was a substantial improvement but also an acknowledgement that the single-scheduler model had hit its limits. The HA scheduler is itself a coordination protocol layered on top of PostgreSQL.

Facetwork has no scheduler. Runners directly claim work from the database. There is no process whose job is to decide what to run next; the decision is made by the workflow evaluator as a consequence of step completion, and the next task is placed in the database immediately. This is structurally simpler and has no single point of failure to replicate.

11.3 The DAG-as-code legacy

Airflow DAGs are Python code. This has been both a strength and a weakness.

Strength: Airflow inherits Python's ecosystem. Data engineers write DAGs using their familiar tools. DAG authors can factor shared logic into Python modules and import them.

Weakness: The DAG is only a DAG *after* the Python code has been executed. The scheduler re-executes the DAG file on every parse cycle to get the current structure. This means:

- DAG parse time affects throughput.
- DAG code runs repeatedly, in the scheduler context, which creates security and resource-isolation concerns.
- DAG code must be importable in the scheduler process, which implies it must be lightweight and free of heavy dependencies.
- Dynamic DAG generation — a common idiom — leads to DAGs that are hard to reason about because the shape depends on runtime state.

Facetwork's FFL is parsed once and the parse tree is the canonical form. The dashboard renders the parse tree directly. The parser is an order of magnitude faster than Python DAG parsing. Dynamic workflow generation is expressed through `andThen foreach` and `catch when`, which are statically analysable.

11.4 The TaskFlow API

Airflow 2.0's TaskFlow API narrows the gap between Airflow's original model and systems like Temporal or Prefect. With TaskFlow, a workflow is written as a Python function, decorated tasks are called as if they were functions, and the dependency graph is inferred from the data flow. This is closer to workflow-as-code in the Temporal sense.

TaskFlow is a genuine improvement but it does not change Airflow's central architectural choices: the scheduler is still central, the execution model is still DAG-based (TaskFlow compiles to a DAG), and the runtime does not support non-DAG control flow. Facetwork's FFL supports conditional dispatch and parallel composition that do not reduce cleanly to a DAG; `andThen when` cases are evaluated at runtime based on the values of preceding facets.

11.5 Live updatability

Airflow supports updating DAG code on the file system and having the scheduler pick up the change. This works for DAG structure changes but not for mid-execution adjustments: a running DAG continues with the DAG as it was when it started. Facetwork's handler registration model allows implementations to be updated mid-execution, with each new task picking up the latest registration.

For workflows that are themselves long-running — a single DAG run taking hours — the difference matters. An operator fixing a bug in Airflow must either wait for the current run to finish (potentially hours) or kill it and restart. In Facetwork, the fix takes effect on the next task; current stages complete, downstream stages use the new handler.

Chapter 12. Jenkins and the Master-Worker Legacy

12.1 Jenkins in brief

Jenkins is a continuous-integration server whose architectural model dates to the mid-2000s. A central controller (historically called "master") holds job definitions, schedules builds, and dispatches them to agents (historically called "slaves"). Pipelines are described in Groovy-based Jenkinsfiles, stored alongside the project source.

Jenkins is not primarily a workflow system — it is a CI/CD controller — but its shape is widely familiar, and many teams use Jenkins for workflow-like tasks that would be better served by a workflow runtime. It represents the centralised-controller tradition in its purest form.

12.2 The single controller

Jenkins has one controller. If the controller is down, no new builds start, no pipeline progresses past its current stage, no one can push a new job. The controller is also the web UI, the job queue, the pipeline interpreter, and the audit log. Every concern of the system is funnelled through the single controller process.

This model is simple. It is also fragile at scale. Large Jenkins deployments accumulate plugins, accumulate job definitions, and accumulate memory pressure until the controller becomes the limiting factor for the whole CI/CD effort.

Facetwork has no controller. Every runner is symmetrical. A fleet of ten runners is neither more nor less critical than a fleet of two; fleet members can be drained, quarantined, and restarted individually. The dashboard is a separate service whose failure does not affect execution. The only shared state is the database, which is already replicated by its own operational team.

12.3 The imperative pipeline model

Jenkinsfiles are imperative Groovy scripts. A pipeline is a sequence of stages, each a block of code. The pipeline describes the execution directly: "do this, then this, then this". There is no separation between pipeline structure and step implementation — the pipeline *is* the code that does the work.

This is the opposite extreme from FFL. FFL describes only the structure; handlers implement the work. Jenkinsfiles conflate the two. The effect is that Jenkins pipelines quickly become intractable: ten-thousand-line pipelines are not rare, and they embody the structure, the implementation, the configuration, and the policy all at once. Refactoring such a pipeline is a months-long project; abandoning and rewriting is the typical path.

12.4 The weakness of the comparison

Comparing Facetwork to Jenkins is, in a sense, a category mistake: Jenkins is a CI/CD system, and Facetwork is a workflow runtime. They solve different problems. But the contrast is instructive because Jenkins embodies a set of architectural choices — centralised controller, imperative pipelines, tight coupling of scheduling and execution — that Facetwork deliberately avoids, and many teams reach for Jenkins when what they actually need is a workflow runtime. Teams running long data pipelines in Jenkins routinely run into:

- Controller memory pressure from large pipeline state.
- Agent-to-controller communication overhead.
- Pipeline code that has grown into an unmaintainable monolith.
- Recovery semantics that amount to "restart from the top".

A workflow runtime with Facetwork's shape is the appropriate replacement for these cases. Teams running short build-and-test cycles in Jenkins are not the target audience; Jenkins is the right tool for that job.

Chapter 13. Evaluation: The OSM Geocoder

13.1 The workload

The evaluation workload is Facetwork's OSM geocoder example, a workflow that:

1. Downloads OpenStreetMap PBF files for a list of regions.
2. Imports each into a local-first disposable PostgreSQL instance, then transfers to the production PostgreSQL server.
3. Extracts multiple feature categories (routes, amenities, roads, parks, buildings, boundaries, population, POIs) from the imported data.
4. Renders statistical summaries and map tiles for each category.
5. Publishes an audit entry per region.

The workload is representative of data-import workflows broadly: heterogeneous I/O, long-running stages, external side effects, fan-out by region.

13.2 Observed behaviour

In a representative run (France + California + Germany, on a three-runner fleet), the workflow exhibited:

- Per-region PBF imports of 90 minutes to 6 hours.
- Staging merges of 15 to 45 minutes.
- Transfer-to-main of 30 minutes to 2 hours.
- Per-category extractions of 5 to 60 minutes.
- Total workflow runtime of approximately 18 hours elapsed, with extensive concurrency.

Failures observed during the run:

- One runner was OOM-killed during a France PBF scan. The lease expired; another runner reclaimed the task and re-ran the PBF scan. The cost was approximately 3 hours of re-work, which is the lower bound for a state-recovery system on a single large stage.
- A transient database connection failure during a staging merge was caught by the workflow's `catch when` block and retried. The retry succeeded on first attempt.
- A handler bug in the `ExtractAmenities` facet (discovered during the run) was fixed, redeployed via `rolling-deploy`, and the failed steps were retried via `Re-run From Here`. Other in-flight extractions continued unaffected.

These three scenarios exercise, respectively, lease-based recovery, FFL error handling, and live-updatable deployment. The workflow survived all three.

13.3 Staged-timeout impact

Before the staged-timeout mechanism (Chapter 8) was added, the same workflow consistently died during the PBF scan for larger regions because the global execution timeout was shorter than the scan duration. The operational response was to raise the global timeout to an unreasonable value (48 hours), which made genuine stalls indistinguishable from active work.

After staged timeouts were introduced, the same workflow runs to completion on the same regions, with the global timeout restored to 4 hours. The PBF scan's stage budget (scaled from file size) keeps the watchdog off the handler during legitimate work; the global timeout still catches genuine stalls. This is the staged-timeout model's intended effect.

13.4 Dashboard observability

Throughout the run, the dashboard rendered:

- Per-step status (pending / running / completed / errored).
- Per-step logs, including handler emission from `ctx.step_log(...)`.
- Active-task counts per runner.
- Heartbeat and stage-budget status per in-flight task.
- `Re-run From Here` / `Retry` / `Reset Block` buttons for operator recovery.

An operator could see, at any moment, which regions were still importing, which extractions had failed, and which recovery action was appropriate. No operator needed to SSH to a runner or tail a log file. This level of first-class observability is a direct consequence of the state-recovery model: every significant event flows through the database and is visible in the dashboard.

13.5 Summary

The OSM geocoder demonstrates that Facetwork's design decisions pay off on a representative long-running distributed workload:

- The DSL encoding of the workflow is concise (approximately 300 lines of FFL for the whole pipeline, versus an order of magnitude more in equivalent Python code).
- The state-recovery model survives handler crashes, runner OOMs, database hiccups, and handler bugs without losing more than one stage's work.
- Live updatability lets operators fix handlers without stopping the fleet.
- Staged timeouts let long stages run while retaining a global safety net.
- The dashboard gives operators the information they need to drive recovery.

Whether the same design would perform similarly on a fundamentally different workload — say, a high-throughput payment processor — is addressed in the next chapter.

Part IV — Closure

Chapter 14. Limitations and Open Problems

14.1 Workloads where Facetwork is not the right tool

Facetwork is designed for long-running, heterogeneous, domain-authored workflows. For workloads with different characteristics, other systems are better:

- **Exactly-once payment processing.** Temporal's determinism and automatic replay-based recovery guarantee that an activity is executed once from the workflow's perspective, which is the right semantic for billable operations. Facetwork's at-least-once execution requires the handler to implement idempotence via external idempotency keys. Both are viable; Temporal's is more automatic.
- **High-throughput job queues.** Celery and Sidekiq are optimised for thousands of short-lived tasks per second. Facetwork's MongoDB-backed claim protocol is adequate at hundreds of tasks per second but was not designed for thousands. A Facetwork port to a higher-throughput atomic store is conceivable but has not been attempted.
- **Heavy business-process workflows.** Camunda and BPMN are the right answer for enterprise processes dominated by human-in-the-loop steps, timer escalations, compensation transactions, and audit requirements specific to regulatory regimes. Facetwork lacks the BPMN interchange layer that these environments often require contractually.
- **Short CI/CD pipelines.** Jenkins, GitHub Actions, and GitLab CI are the right tools for build-test-deploy cycles. Facetwork is over-engineered for that use case.

Acknowledging these boundaries is important. A workflow system that claims to be universally best is generally a workflow system that is actually best only for its authors' personal workloads.

14.2 Open problems in the Facetwork design

The Facetwork design has open problems and the honest account of them is part of the thesis.

Idempotence enforcement. Facetwork relies on handler authors to make external effects idempotent where needed. There is no compiler check, no runtime enforcement, no default mechanism. A principled solution would be a typed `@idempotent` annotation on handlers, a framework for idempotency keys, and a runtime check that flags non-idempotent facets that are being retried. This has not been implemented.

Schema migration across handler versions. When a handler's return schema changes, in-flight workflows may have steps whose persisted state is in the old schema. Facetwork's current practice is to avoid breaking schema changes by adding fields but not removing or renaming them. A more principled solution would be typed schema migrations as first-class FFL constructs; this is future work.

Cross-workflow dependencies. A primitive for cross-workflow composition already exists, and it is more elegant than the term "limitation" suggests: **any workflow can be invoked from another workflow exactly as if it were an event facet**. A workflow is, after all, just a facet designated as an entry point; its typed parameters are the workflow's inputs, its `yield` is the workflow's output, and its `andThen` body is its computation. From the caller's perspective there is no syntactic distinction between calling a sub-workflow and calling a leaf event facet:

```
workflow RegionalReport(region: String) => (report: Report) andThen {
  imported = ImportRegion(region = $.region, force = false)   // another workflow
  enriched = EnrichWithPopulation(source = imported)          // leaf event facet
  stats    = RegionalStatistics(data = enriched)              // another workflow
  yield RegionalReport(report = stats.report)
}
```

The data-dependency semantics of §4.4 extend uniformly: a sub-workflow reached through a parallel branch runs concurrently with its siblings, and downstream facets that reference its result block until it completes. The block-mediated advancement of §5.8 applies at every level of the call tree — each sub-workflow has its own evaluator and its own continue-message discipline — so composed workflows inherit the same lock-free coordination as monolithic ones. This makes workflows first-class reusable units: a team can publish a workflow alongside its event facets, and downstream teams can consume it by name, substituting implementations at the handler level without touching the consumer's FFL.

A concrete demonstration of this came together as the standalone-package pattern of §6.9 matured. The `fwh_anthropic` package declares `anthropic.messages.CreateMessage` and a small companion `anthropic.compose.DocumentQA` workflow (upload a file via the Files API, ask Claude about it via the Messages API — the canonical retrieval-augmented-generation shape). The framework's in-repo `research-agent` example was then ported to consume `anthropic.messages.CreateMessage` from `fwh_anthropic` rather than declaring its own prompt-block facets — the domain workflow plans, decomposes, gathers, analyses, drafts, and reviews entirely by issuing `CreateMessage` calls against system prompts that demand JSON, and parsing the responses through small typed `Parse<Schema>` event facets. From the FFL author's perspective there is no syntactic distinction between `anthropic.messages.CreateMessage` and any local facet; the discovery is operational (which runner has the matching handler) rather than linguistic. This is precisely the cross-package composition that §4.10 argued the language would enable; eight standalone packages and one cross-package consumer later, it is no longer hypothetical.

What remains genuinely under-specified is the *cross-instance* case: when two independently-triggered top-level workflow instances need to share state, await each other's completion based on runtime-discovered identities, or serialise access to an external resource. Composition via facet-like invocation handles the static case well; the dynamic, instance-to-instance case — analogous to `signal`, `query`, and `waitForExternalEvent` primitives in Temporal — is future work.

Global observability across the fleet. The dashboard renders per-workflow and per-server state; it does not yet render fleet-wide metrics (total throughput, queue depth, aggregate lag, SLO adherence). Grafana integration covers database-level metrics well but workflow-level metrics less well. A proper fleet dashboard is future work.

The MongoDB dependency. Facetwork is tightly coupled to MongoDB's atomicity primitives. A port to PostgreSQL (using `SELECT ... FOR UPDATE SKIP LOCKED`) is plausible and would widen Facetwork's applicability, but the porting work is non-trivial because the claim protocol, the partial unique index, and the heartbeat model all assume MongoDB semantics.

14.3 Scale limits

Facetwork has been exercised with:

- Fleets of up to twelve runners across three physical hosts.
- Workflows of up to several hundred steps per run.
- Concurrent run counts of up to approximately one hundred.
- Per-step task sizes from milliseconds to hours.

Beyond this scale, I do not have empirical data. The claim protocol's throughput is bounded by MongoDB's `find_one_and_update` rate, which on modest hardware is in the hundreds to low thousands per second. The dashboard's rendering is bounded by the cursor paging; pages of ten thousand steps become slow. Neither bound has been hit in production use, but they exist.

Scaling past these bounds is plausible but would require either partitioning the task collection (so that each runner only reads a shard), sharded MongoDB deployments, or optimistic claim protocols that batch work per call. None of these changes affect the design arguments of this thesis; they are operational refinements.

14.4 Issues identified and resolved in cross-package validation (note added in proof)

The cross-package demonstration of §14.2 — porting `research-agent` to consume `anthropic.messages.CreateMessage` from `fwh_anthropic` — surfaced three latent runtime issues that the original OSM-only evaluation had not exercised. All three were corrected; they are worth recording briefly both because they illustrate the kinds of bug the state-recovery model makes visible and because the fixes themselves form part of the current codebase.

The first was a self-recursion in the AgentPoller's `_safe_save_task` retry wrapper — the function called itself rather than the underlying `persistence.save_task`, so every transient save failure spiralled into a recursive retry loop. The bug had been latent because production MongoDB rarely returns transient errors; it surfaced when MongoDB hiccuped under the multi-runner Compose stack of §6.9 and the dashboard's log stream made the recursion visible immediately.

The second was a gap in `resume_step`, the event-driven cascade that propagates child-completion notifications up through enclosing blocks. The cascade marks the parent block as dirty when a child progresses, but it did not also re-queue *sibling* steps of the dirtied block that were parked at `FacetInitBegin` with `_StepNotReady` raised against a cross-sub-block step reference. The original OSM workflow's fan-out happened entirely inside `andThen foreach` blocks, where iteration boundaries forced the right re-queue order by accident; the research-agent's three parallel `g0 = GatherSources(...) andThen { f0 = ... }; g1 = ...; g2 = ...; synth = SynthesizeFindings(all_findings = [f0.findings, f1.findings, f2.findings])` pattern exposed the gap. The fix re-queues stuck siblings whenever a block becomes dirty, finding them by both `block_id` and `container_id` so that block-level children of a workflow root are included.

The third was an over-strict completion check in cross-block reference resolution. The `get_completed_step_by_name` helper required `step.is_complete`, which deadlocked the standard inline-andThen back-reference pattern `g0 = Gather(...) andThen { f0 = Extract(sources = g0.sources) }`: `g0` cannot reach `STATEMENT_COMPLETE` until `f0` finishes (the inline block is part of `g0`'s own lifecycle), and `f0` cannot start until `g0.sources` resolves. The fix accepts any step whose `attributes.returns` has been populated — which happens at the EventTransmit transition, well before STATEMENT_COMPLETE — preserving the §4.4 data-flow semantics without the false circularity.

Each of these three issues has a regression test in the runtime test suite, exercising the patterns that surfaced them (statement-level fan-out, foreach iteration, inline-andThen back-references). Together they advance the §13.5 evaluation claim — that the design survives crashes, hiccups, and handler bugs without losing more than one stage's work — by an additional point: the state-recovery model also makes *internal* runtime bugs diagnosable through the dashboard rather than through gdb.

Chapter 15. Conclusion

15.1 What has been argued

This thesis has argued for a specific position in the distributed workflow design space:

- Workflows should be described in a **typed DSL**, not in general-purpose code and not in graphical standards. The DSL separates topology from implementation, enables compile-time topology checks, and supports domain-expert authorship.
- Work should be coordinated through **lock-free atomic operations** over a shared data store, not through a central scheduler or a broker or a replay service. This eliminates the single point of failure, reduces operational complexity, and scales proportionally with the underlying store.
- Workflow state should be **persisted as a mutable graph**, not as an immutable event log. This allows step-level recovery, drops the determinism constraint, and accommodates the non-deterministic, side-effect-heavy nature of realistic handlers.
- Fleet operations should be **live and granular**. Handlers should be registerable at runtime. Runners should be drainable and quarantinable individually. Timeouts should be composable, with per-stage budgets nested inside global ceilings. Deployments should roll without downtime.

Each of these claims has been illustrated by Facetwork's design and evaluated against the major alternative systems in the space (Temporal, Camunda, Airflow, Jenkins). The design choices reinforce rather than undercut each other: the DSL's separation of topology and implementation only matters if handlers can be changed live; the state-recovery model only scales if recovery is step-level; the step-level recovery only works if the topology graph is statically available. Facetwork arrives at a coherent point where all four are simultaneously true.

15.2 What has not been claimed

I have not claimed that Facetwork is better than every alternative on every workload. I have claimed that on a specific class of workload — live long-running domain workflows — Facetwork is structurally better than each of the major alternatives, for reasons I have attempted to justify rather than assert.

I have not claimed that Facetwork's lock-free protocol is novel in a deep sense. It is a straightforward application of atomic document operations, which have been available in databases for decades. The novelty, such as it is, is in the *combination* with the other design elements: the DSL, the state recovery, the live updatability, the staged timeouts.

I have not claimed that FFL is the best possible DSL. It is one reasonable point in the design space; others exist; the design choices I defended are not the only defensible ones. A proper comparative evaluation of workflow DSLs is future work.

15.3 Where the design goes next

Four directions seem most promising for further development.

First, typed idempotence. Adding `@idempotent` annotations to handler registrations, with compile-time and runtime checks, would eliminate one of the real costs of state-recovery semantics. The groundwork exists; the typing discipline is well-understood; the implementation is straightforward.

Second, cross-workflow coordination. A workflow that waits on another workflow's completion, or that blocks until a condition holds in a shared external store, is expressible today but not elegantly. A first-class `await` construct in FFL, coupled with a runtime signal mechanism, would clean this up.

Third, federated fleets. The current design assumes one fleet per MongoDB instance. Supporting federation — where multiple fleets share workload boundaries but coordinate through fleet-boundary primitives — would extend Facetwork's applicability to multi-site deployments without compromising the lock-free local claim protocol.

Fourth, AI authorship and sealed skills. The thesis frames FFL authorship as a collaboration between domain programmers and service-provider programmers. In practice, during the development of Facetwork, an AI agent produced most of both layers under human direction. The implications are substantial and deserve explicit treatment.

The argument is not the popular one that workflow systems are "deterministic and therefore safe for non-deterministic AI." Facetwork is not deterministic in any of the strong senses the term usually carries (deterministic replay, end-to-end reproducibility); it is deterministic only in its *topology* and *orchestration*. What workflows actually offer agentic systems is **procedural consistency with leaf-level variability**: the same DAG, the same typed interfaces, the same recovery semantics, regardless of which LLM backs each facet. Handlers remain non-deterministic at their leaves; the scaffold around them is reproducible.

If AI agents become the primary authors, the operational shape shifts. The human's role becomes directing intent, reviewing generated artifacts, and curating a library of approved ones. A generated workflow becomes a **sealed skill** — FFL source plus pinned handler registrations plus a contract plus provenance metadata (model, prompt, reviewer, tests) plus a content-addressable identifier plus a signature. Sealed skills are immutable, discoverable, and composable; they form a registry that AI agents query before regenerating. The operational cycle is **AI generates, human reviews, skill seals, fleet executes**.

The design changes that follow are additive rather than relaxing. Effect annotations (`@pure`, `@idempotent`, `@llm`, `@external-side-effect`) on every facet. Refinement types and sum types to tighten return contracts. Pre- and post-conditions verifiable at compile time. Schema inference from examples to stop AI hallucination of shapes. Two-phase generation (typed stubs first, implementations second). Structured docstring annotations (`@purpose`, `@failure-modes`) cross-checked against the code. A content-addressable skill registry with a discovery protocol. Regeneration rendered as semantic diffs for human review. Auto-generated property-based tests as a precondition of sealing. Confidence markers on generated output. Counterfactual replay (re-running a workflow with different LLMs behind a chosen step). Removal of the `script python` escape hatch and of any remaining un-checkable surface.

Two tempting directions must be resisted. The first is making FFL unreadable to humans on the grounds that AI reads and writes it — the machine-readable form is authoritative, and giving up human inspectability forfeits audit, debugging, and ontological portability for a benefit (AI-optimised syntax) whose absence costs essentially nothing. The second is relaxing FFL's restrictions on the grounds that AI can handle more complex grammars — the smallness of FFL serves the *compiler and tooling*, not the author, and the research literature on synthesis-friendly languages consistently argues for *more* constraint, not less, when the author is a program synthesiser. Both temptations mistake what FFL's simplicity is for.

The extended treatment — with concrete feature proposals, detailed arguments against the tempting wrong turns, and a full discussion of the implications for the handler model — appears in the companion document `ai-authorship.md` alongside this thesis. That document is the authoritative record of the AI-authorship design direction; this subsection is its summary.

Each of the four directions is an evolution within the thesis's design position, not a departure from it. Each preserves the DSL, the lock-free coordination, the state-recovery model, and the live updatability. Each extends the applicability of the whole.

15.4 Closing

The point of a PhD thesis is not to announce that its author has had the last word. It is to argue a defensible position clearly enough that subsequent work can either build on it or refute it. I have attempted the former. Whether I have succeeded is, in the nature of these things, for the reader to decide.

I close with a concrete claim. A team charged with building a new distributed workflow system today, for a workload that matches the live long-running domain workflow profile, should:

1. Write a typed DSL for its workflows. Resist the temptation to reach for a general-purpose language. The cost of the DSL is paid once; the cost of unbounded code-as-workflow is paid forever.
2. Coordinate work through atomic operations on a shared store. Resist the temptation to introduce a coordinator. Most of the alleged benefits of a coordinator are replicable through careful atomic-store design; most of the costs are structural.
3. Persist state, not history. Resist the temptation to adopt replay-based recovery. For workloads with long, side-effectful stages, state recovery is simpler, faster, and more ergonomic.
4. Treat fleet operations as first-class. Build drain, quarantine, and rolling-deploy primitives from day one. The alternative is to grow them reluctantly under production pressure.

Facetwork is one expression of these principles. I hope this thesis has made the case that they are general.

References

[Agh86] Agha, G. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.

[Arg] Argo Workflows. https://argoproj.github.io/workflows/

[Arm03] Armstrong, J. *Making reliable distributed systems in the presence of software errors*. PhD thesis, Royal Institute of Technology, Stockholm, 2003.

[Bur06] Burrows, M. The Chubby lock service for loosely-coupled distributed systems. *OSDI*, 2006.

[CCH+10] Chambers, C., Raniwala, A., Perry, F., Adams, S., Henry, R., Bradshaw, R., Weizenbaum, N. FlumeJava: Easy, efficient data-parallel pipelines. *PLDI*, 2010.

[Cad] Cadence Workflow. https://cadenceworkflow.io/

[CWL20] Crusoe, M.R. et al. Methods included: Standardizing computational reuse and portability with the Common Workflow Language. *Communications of the ACM*, 65(6), 2022.

[DLS88] Dwork, C., Lynch, N., Stockmeyer, L. Consensus in the presence of partial synchrony. *Journal of the ACM*, 35(2), 1988.

[DTP+17] Di Tommaso, P., Chatzou, M., Floden, E., Barja, P., Palumbo, E., Notredame, C. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 2017.

[FLP85] Fischer, M., Lynch, N., Paterson, M. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2), 1985.

[GC89] Gray, C., Cheriton, D. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. *SOSP*, 1989.

[GKR+19] Goldstein, J., Abdelhamid, A., Barnett, M., Burckhardt, S., Chandramouli, B., Gehrke, J., Hunter, R. A.M.B.R.O.S.I.A: Providing performant virtual resiliency for distributed applications. *VLDB*, 2020.

[GSR+15] Gog, I., Schwarzkopf, M., Grosvenor, M., Pietzuch, P., Hand, S. Musketeer: all for one, one for all in data processing systems. *EuroSys*, 2015.

[Hew73] Hewitt, C., Bishop, P., Steiger, R. A universal modular ACTOR formalism for artificial intelligence. *IJCAI*, 1973.

[HKJR10] Hunt, P., Konar, M., Junqueira, F.P., Reed, B. ZooKeeper: Wait-free coordination for internet-scale systems. *USENIX ATC*, 2010.

[IBY+07] Isard, M., Budiu, M., Yu, Y., Birrell, A., Fetterly, D. Dryad: Distributed data-parallel programs from sequential building blocks. *EuroSys*, 2007.

[KR12] Köster, J., Rahmann, S. Snakemake — a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19), 2012.

[Kub] Kubernetes Jobs. https://kubernetes.io/docs/concepts/workloads/controllers/job/

[Lam98] Lamport, L. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2), 1998.

[OMG11] Object Management Group. Business Process Model and Notation (BPMN), Version 2.0. Specification, 2011.

[OO14] Ongaro, D., Ousterhout, J. In search of an understandable consensus algorithm. *USENIX ATC*, 2014.

[ORS+08] Olston, C., Reed, B., Srivastava, U., Kumar, R., Tomkins, A. Pig Latin: A not-so-foreign language for data processing. *SIGMOD*, 2008.

[Per17] Perham, M. Sidekiq: Simple, efficient background jobs for Ruby. https://sidekiq.org/

[Sol21] Solem, A. Celery distributed task queue. https://docs.celeryq.dev/

[Tem] Temporal Workflow. https://temporal.io/

End of dissertation.